

ECE 4972/4973: Report

Enhanced Baseball Training

Virtual Reality Simulator to Improve Traditional Hitting Practice

Dmitrii Kapranov, Amiri Prescod, Michael Kokolis, Christopher Powers, Julian Frank



VILLANOVA
UNIVERSITY
IGNITE CHANGE. GO NOVA.®

Abstract

Baseball players are constantly looking for optimized methods of training and receiving metric feedback. The purpose of this project is to create a system in which baseball players can receive efficient data related to the swing path of the bat via a 3D rendering. By combining two existing methods popular in gathering baseball-related data, camera tracking and motion measurements, the batter receives smooth feedback data involving the motion of the bat. Using an NVIDIA Jetson Orin Nano GPU and an IMU sensor, the positional data of the batter's hand will be merged with the motion data from the IMU to create a 3-dimensional rendering of the batter's swing plane. This rendering will include metrics such as maximum tangential velocity, moment of impact, and time through the pitch plane. This system will allow baseball players to not only view each swing with great detail, but also analyze and train based on the gathered data at new levels.

Contents

Abstract	2
Chapter 1: Introduction	5
Point-Of-View Statement	5
Problem Statement	5
Ethnographic Research	5
Brainstorming	7
Three Best Ideas	7
Chapter 2: Design	9
Constraints and Criteria	9
Decision Matrix	9
Decision	10
Solution Description	10
Chapter 3: Project Technical Details	13
Project Overview	13
Specifications	14
Testing Summary	14
Testing Plan	15
Schematics and Drawings	18
Client Materials	20
Chapter 4: Broader Impacts	20
Public Health, Safety, and Welfare	20
Global, Cultural, and Social	20
Environmental	20
Economic	20
Engineering Standards	21
Risk Assessment	21
Chapter 5: Project Management	22
Schedule	22
Personnel	22
Parts and Components	22
Skills and Tools	23
Budget	23

Chapter 6: Achievements and Conclusions	24
Achievements	24
Conclusions.....	24
References	25
Appendix A: Ethnographic Research.....	26
Appendix B: Code/Schematics/Etc.	27

Chapter 1: Introduction

This chapter provides the rationale for developing a virtual reality (VR) baseball training simulator and presents the ideas that emerged from the team's brainstorming process. The chapter starts with a Point-Of-View statement specifying the intended user and their need for such simulator. Next, a problem statement is offered that shares the scope of the project in detail followed by the findings of ethnographic research conducted to identify existing technologies and the feasibility of the anticipated project.

Point-Of-View Statement

Baseball and softball batters as well as their coaches need a virtual reality tool to optimize their training process because traditional practices do not efficiently provide necessary feedback metrics.

Problem Statement

Several metrics about a baseball or a softball swing can be used to help batters maximize effectiveness of hitting performance. Players often get only a few good pitches to swing at per game; thus, they need to optimize each attempt. Small adjustments that can be made during training can make a significant difference in at-bat outcomes.

The objective of the project is to create a way for players and coaches to easily identify the technical shortcomings of swings to make fast and accurate corrections. This goal can be achieved by creating a swing visualization tool and computing relevant swing metrics including:

- bat speed (specifically barrel speed at contact)
- attack angle (the angle of the bat relative to the ground at point of contact)
- time to contact (time between start of swing and contact)
- swing plane efficiency (how long the barrel stays on plane in the hitting zone)
- rotation speed (angular velocity of hands/bat)

With such information available, players and coaches can immediately identify and tweak flaws. Hence, the intention of the product is to provide a scheme to compute concrete and accurate feedback metrics and give players ability to visualize their swings for further analysis. The project consists of collecting data about location and motion of the bat using quality cameras and application specific inertial measurement unit (IMU). The data is then uploaded into a state-of-the-art extended reality headwear to produce a virtual swing rendering.

Ethnographic Research

Before designing the simulator, the team conducted ethnographic research consisting of the following elements: (1) a literature review that evaluated health concerns related to VR and its relative effectiveness compared to traditional training; (2) observation of a division 1 (D1) college baseball game to gauge the current technologies utilized by athletes, and (3) interview with an engineer with experience in developing sports technology to determine potential challenges and useful technical resources for realization of the intended project.

This research yielded two Institute of Electrical and Electronics Engineers (IEEE) standards that address concerns associated with use of VR technology [1][2]. These standards were followed throughout the development process to ensure safe implementation of the project. Additionally, several articles concluded that VR is in fact superior to current training philosophies. In [3], 4 of 5 subjects showed significant improvement in their swings just after 5 days of using VR technology. The study in [4] showed that VR gaming is just as effective in improving hand eye coordination, reaction time, and agility as real sports like table tennis. The full raw preliminary research can be found in Appendix A.

After attending a D1 college baseball game, it was discovered that technology is utilized to only provide pitch velocity metric to players during pre-game batting practice which is only indirectly helpful to batters. So effectively, teams competing at one of the highest-level baseball leagues in the country do not have any technological assistance in their training. This lack of swing analysis during preparation was reflected in the game as players could not make mid game adjustments leading to poor swings and contributing to a low scoring game dominated by pitching and defense.

The team interviewed Mr. Edward Dougherty, a founding member of the Novation Tech LLC – the company that specializes in proof-of-concept products in the sports and entertainment industry. In his professional experience, Mr. Dougherty worked on many object tracking projects and commercialized PITCHvr application that renders baseball swings. During that experience, he has figured out critical metrics to consider when analyzing baseball swings, including:

- Explosiveness – peak acceleration of the swing
- Snap – rate of angular acceleration of the bat
- Whip – bat velocity at contact
- Burst – time to reach peak bat speed
- Surge – time to reach peak acceleration
- Jolt – rate of change of acceleration
- Impulse – force produce by the bat at contact

Mr. Dougherty also addressed challenges that arise when working with cameras for object detection purposes such as camera speed, zoom, and resolution and emphasized importance of good lighting conditions. Additionally, he brought attention to the frame rate of the cameras as well as various physical setup considerations. Speaking of the physical setup, Mr. Dougherty recommended that one camera be placed directly above and the other directly in front of the batter, enabling cameras to effectively track the bat's vertical and horizontal displacements. The team investigated that proposal during the implementation step of the project. Lastly, Mr. Dougherty provided input as to which programming languages should be utilized. More specifically, Mr. Dougherty mentioned that language does not often matter as they all have similar capabilities. However, he said that C and Python have more open-source information which would make the learning curve less steep.

Brainstorming

The initial brainstorming involved a wide spectrum of ideas ranging from practical possibilities to purely speculative considerations. Several ideas revolved around adding more sensors for additional tracking and data collection including sensor-equipped bats, wearable devices, and camera schemes to capture body and ball movement in addition to bat tracking. Under another umbrella of ideas were the real-time visualization inquiries that consisted of holographic and 3D renderings with heat maps and live performance assessment in the headset. Multiple proposals pushed for AI-driven feedback such as loading coaching insight based on swing outcome. Finally, some concepts aimed at the entertainment aspect via adding game-like atmosphere, implementing leaderboards, and enabling multi-player modes. Overall, most of the brainstorming was rather unfeasible but it allowed the team to gain more insight into what kind of metrics can be sought after during the development of the simulator. The most reasonable ideas from the brainstorming contained the physical location of the cameras to make data collection more accurate. The best three of those are analyzed in the next section.

Three Best Ideas

After the original brainstorming session, the efforts were narrowed down to deciding how cameras should be set up with respect to the batter. The intent of the cameras is to continuously capture 3D position of the bat in space, so the goal was to minimize instances during which the bat tracker would not be picked up by either camera. This is important because accurate position data from the cameras allows the computation of orientation values and the rest of the feedback metrics.

The first idea is shown in Figure 1. The idea included two cameras located 45 degrees from the home plate on either side of the batter. This idea is best for analyzing rotational mechanics, but it allows for the possibility of losing the bat path (i.e. during follow-through).

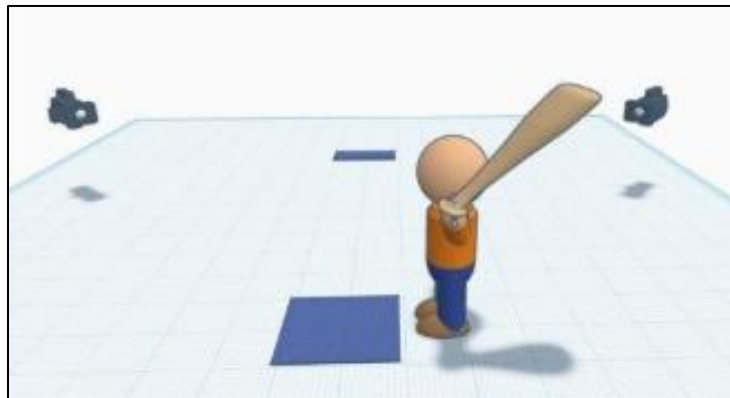


Fig. 1. Physical set up with cameras at each side of batter

Figure 2 presents the second idea where one camera would be located around the pitcher's mound and the other directly above home plate. This option is best for swing path analysis and would give the same quality of data for left-handed and right-handed hitters without having to readjust the setup. However, the overhead camera may not have a guaranteed clear view if the batter leans forward, for example. Additionally, parallax errors are possible.

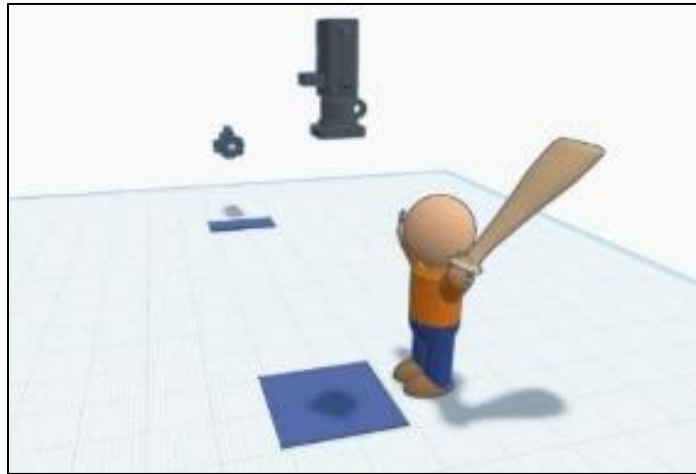


Fig. 2. Physical set up with cameras in front and above the batter

The third and final idea is demonstrated in Figure 3. For the left-handed batter, the first camera would be located back left of the batter while the second camera would be in the front right. This option is best for tracking depth-based movements such as bat path through the strike zone but suffers from less unique data due to the staggered angles. Additionally, the setup must be adjusted based on the handedness of the batter.

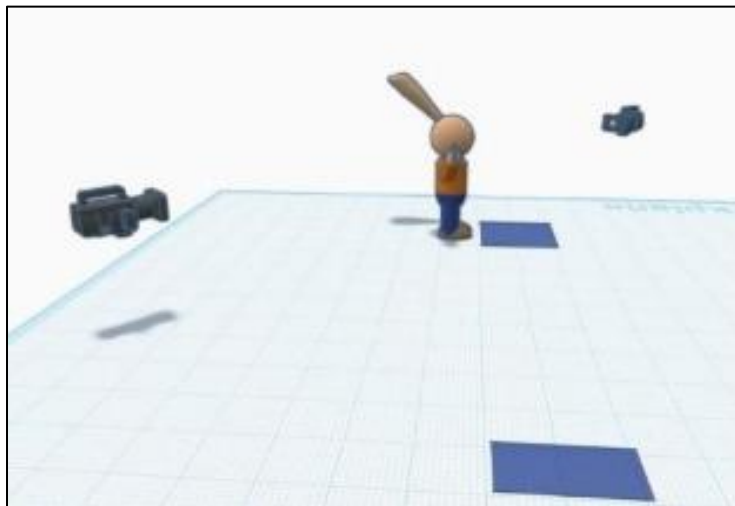


Fig. 3. Physical set up with cameras located diagonally from each other. For left-handed batter (as shown), respective camera locations are back left and front right with respect to the batter.

Chapter 2: Design

This chapter presents the analysis of three proposed solutions. First, constraints and criteria are listed based on objectives of the project. Next, three best ideas are given a score based on how well they align with desired characteristics. Finally, the best solution is chosen and described.

Constraints and Criteria

After considering three ideas from the previous section, constraints and criteria were developed to make the selection of the final product. Main considerations in choosing constraints and criteria were user-oriented, ensuring that the setup of the product is fast and easy while desired data collection is accurate.

1. **Constraint 1: Ease of setup adjustment for a specific batter** (considers the amount of effort necessary to vary the setup based on batter handedness, height, and level of play)
2. **Constraint 2: Accurate statistics** (evaluates the accuracy of position data measured by the cameras)
3. **Constraint 3: Camera safety** (evaluates the probability of damage to cameras during training)
4. **Criterion 1: Full swing capture** (determines whether the given physical setup is capable to capture the entire swing path from back swing to follow-through)
5. **Criterion 2: Compatible with any bat** (determines whether the setup can be used with any bat including size and material)
6. **IEEE Standard for Head-Mounted Display (HMD)-Based Virtual Reality (VR); IEEE 3079 [1]**

Decision Matrix

Evaluation of the three ideas is shown in Table 1 via a decision matrix. Importantly, all constraints were given a score ranging from 0-2. Additionally, each design was determined to be either criterion-compliant (Y) or not (N). The justification for each score seen in the table is provided in the following section.

Table 1. Decision Matrix to Select the Best Physical Setup

Solutions	Setup Adjustment	Accuracy	Camera Safety	Full Swing Capture	Bat Choice Compatibility	Total
Camera 1: Behind pitcher Camera 2: Above home plate	1	2	1	Y	Y	4
Camera 1: Behind batter Camera 2: Side of batter	0	1	2	Y	Y	3
Cameras on opposite sides of batter at 45 degrees	2	1	1	N	Y	4

Key: Constraints ranked 0-2, Criteria N=no, Y=yes

Decision

The optimal camera setup for the project is Option 1 with camera 1 behind the pitcher's mound and camera 2 directly above home plate. This setup scored best with a total of 4 points, providing balanced results across setup adjustments and camera safety while meeting the criterion of full swing capture. In this setup, the camera behind the pitcher's mound accounts for the up and down motion of the bat, and the other camera takes care of bat moving forward and sideways. Given the objective to precisely analyze baseball swing mechanics, accuracy is the most critical component to consider when choosing a camera configuration. This setup received the highest score of 2 in that category because of swing coverage and ability to bring the cameras closer if necessary.

By comparison, the two alternative configurations scored 3 points each, suffering from tradeoffs that made them less ideal. Option 2 with camera 1 behind the batter and camera 2 on the side of the batter configuration is the safest for the cameras. However, this option needs to be constantly adjusted for differently handed hitters. Similarly, Option 3 with cameras on either side of the batter is the easiest to set up. However, it has poor accuracy due to the possibility of cameras losing sight of the bat during back swing and follow through. Hence, this option failed to miss the full swing capture criterion and was disqualified from being considered as a viable implementation option.

Solution Description

The final product combines camera and IMU tracking schemes that collect and compute data to efficiently summarize important swing metrics such as position, velocity, acceleration, and orientation. Data from two schemes is then synchronized in time domain using cross-correlation procedure. Once synchronization is complete, the data can be input into the state-of-the-art Kalman filter to produce a virtual rendering of the swing. To encompass the full range of the swing, the cameras are placed directly above and in front of the home plate. The batter wears an IMU transmitter in a small square casing that is attached to the batter's top hand via Velcro. Glued to the top of the casing is spherical object with four high contrast colors (green, blue, pink, and yellow) to be picked up by two Microsoft Azure Kinect cameras as shown in Figure 4. The cameras use their embedded depth sensing capabilities to measure position of the colored spheres in three dimensions. From that data, the quaternion coordinates are computed for orientation measurements. Additionally, the tracking programs used by the cameras are normalized to a single point to ensure data consistency. The IMU part of the design wirelessly transfers local acceleration and rotation data from the above-mentioned transmitter to a receiver that is programmed to store data in a designated location. The IMU transmitter and its casing as well as the receiver appear in Figure 5. For better processing speed, data from both schemes is transferred to NVIDIA Jetson Orin Nano Graphics Processing Unit (GPU) enabling faster and higher quality feedback to the player. The block diagram of the entire project is shown in Figure 6.

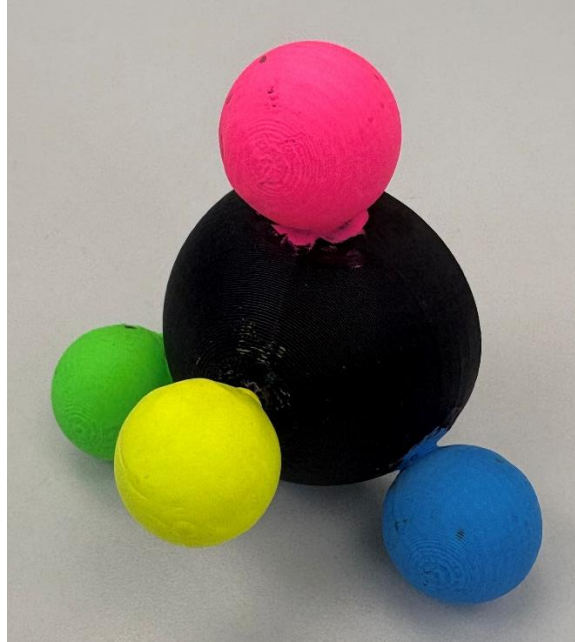


Fig. 4. Trackable Spherical Object for the Camera Scheme

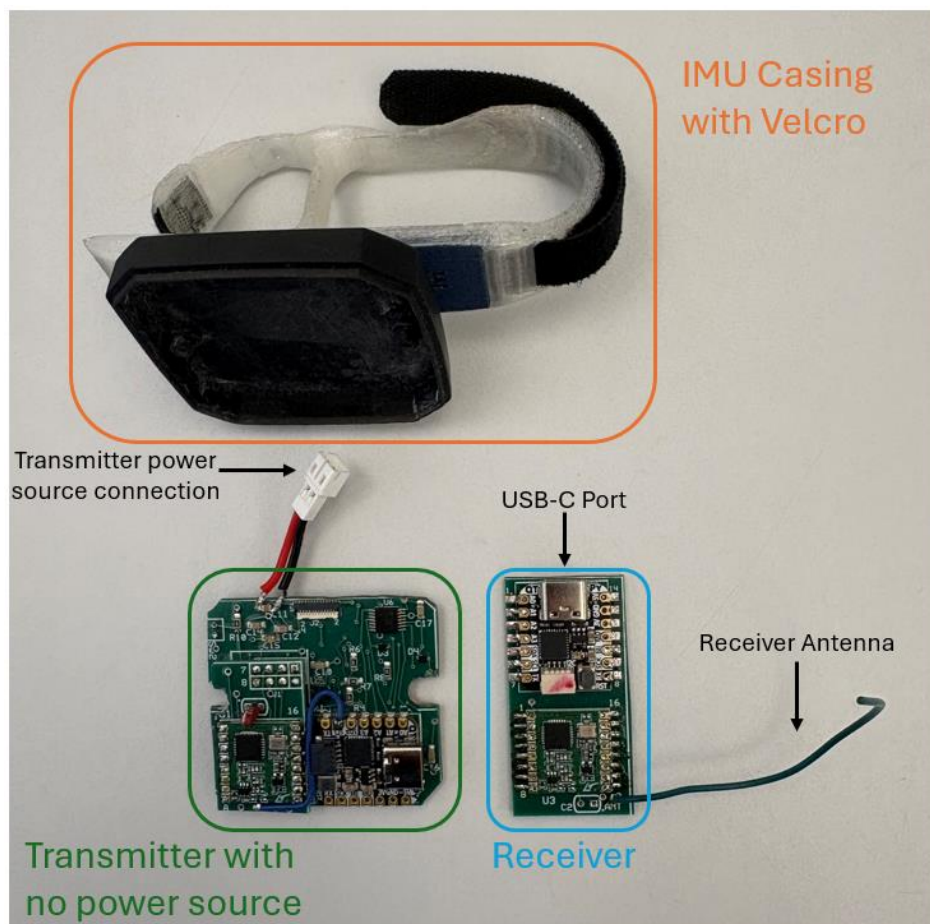


Fig. 5. IMU Scheme Hardware Parts including the Casing, Transmitter, and Receiver

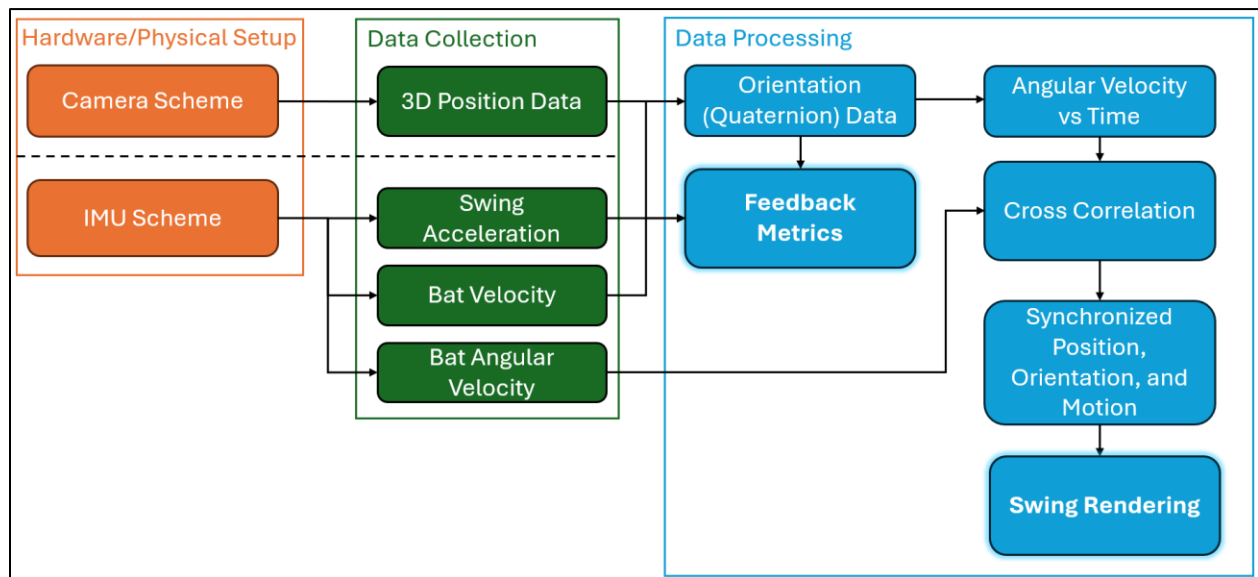


Fig. 6. Baseball Training Simulator System Design Block Diagram

As one can induce from the block diagram, there are three major subsystems in the project, namely hardware, data collection, and data processing. For the hardware, the team configures the cameras and 3D-prints the trackable spherical object. Additionally, the team solders the power source for the IMU transmitter and obtains cables to connect receiver of the IMU to either a computer or a GPU. In data collection, the programs are written in Python to measure and tabulate data from each of the hardware schemes. The position data from the cameras is continuously published over a ZeroMQ publish/subscribe socket and stored in a JSON database for external access. Meanwhile, the IMU data is stored in Excel with clear formatting and labeling. Moving on to data processing, more Python and one MATLAB program are written to compute orientation metrics from the collected position data. These quantities are stored as quaternion coordinates. They are then used for determining angular velocity of the bat in time domain using known linear algebra transformations. In the IMU scheme, angular velocity is measured directly by the tracker. So, two angular velocity data streams are synchronized in time domain using cross correlation techniques. Once the data is aligned, it is used as the input to the Kalman filter generating a swing rendering in the VR space.

Chapter 3: Project Technical Details

Project Overview

Figure 7 shows the final block diagram of the project. Overall, the team managed to stay with intended goals, so there were no radical changes to the original block diagram seen earlier in chapter 2 on Figure 4. As the team was developing the project, new design goals popped up which created additional specifications which will be explained in the next section. Despite not having many changes to the overall design plan, the team was unable to create an extended reality swing rendering in the allotted time due to underestimating the amount of time it would take to configure hardware and achieve accurate measurements to start data processing work. Hence, there was a discrepancy between planned and realized project timelines. In the following sections, all the achieved stages are shown with their corresponding tests and results.

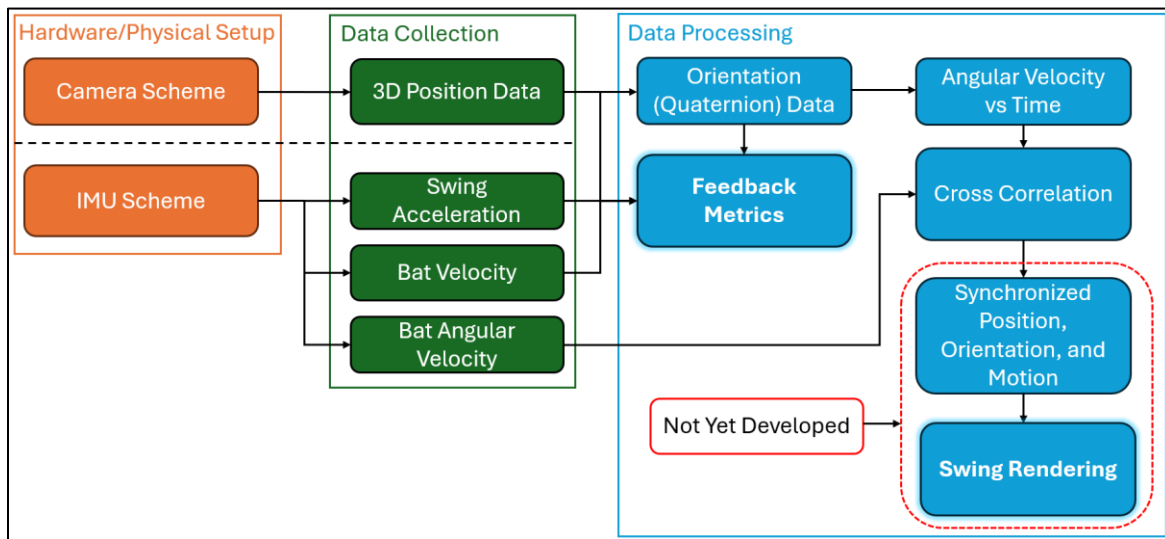


Fig. 7. Enhanced Baseball Training Final System Design Block Diagram\

Specifications

As the team started building the project, new design goals were established. The technical specifications are summarized in Table 2 with corresponding explanations.

Table 2. Technical Specifications

#	Specification	Explanation
1	Criterion 1	data capture rate of at least 55 frames per second
2	Criterion 2	system tracker must be low weight (under 5 oz)
3	Criterion 3	tracking works at 10 feet distance from batter
4	Constraint 1	3D position is continuously captured by cameras and accurate
5	Constraint 2	camera and IMU data are synchronized in time domain
6	Specification 1	base and trackable spheres are high contrast
7	Specification 2	IMU powered by a battery
8	Standard 1	<i>IEEE 1588: Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems [5]</i>

Following is expanded reasoning for each specification above:

- 1) Each camera has a data capture rate of 30 frames per second (fps). By interlacing frames, the rate of data collection is maximized at 60 fps. To allow for occasional stream corruption and intermittent processing delays, the threshold is lowered to 55 fps.
- 2) Under 5 ounces, added weight essentially has no effect on the swing.
- 3) At 10 feet, the probability of impact from a batted ball is sufficiently low to run the project. However, shielding boxes or similar are still recommended to protect cameras from direct impact.
- 4) The entire data processing part of the project is dependent on reliable position data. Hence, it is important to continuously capture data with the cameras when using the product.
- 5) To produce a virtual swing rendering, synchronized data needs to be loaded into Kalman filter.
- 6) For accurate color-based detection, spheres need to be easily identifiable by the cameras.
- 7) IMU transmitter must be wireless. Hence, there is a need for a battery.
- 8) The standard verifies that cameras and GPU are well-equipped for high-speed data transfer and synchronization by looking at the offset and drift values when running both components.

See the following section for more information on each technical specification.

Testing Summary

Table 3 summarizes the tests carried out to ensure all technical specifications from the previous section have been met. For each test, passing criteria as well as corresponding technical specifications are listed.

Table 3. Testing Plan Summary

Test	Passing Criteria	Technical Specification(s)
Camera Test	• Data collection rate is at least 55 fps • 3D Position coordinates are displayed in terminal as the project is running	Criterion 1, Constraint 1, Specification 1

	• Position coordinates are accurate within 5 cm	
Data Test	• IMU data collection is performed wirelessly • Average per cent difference between aligned data points from IMU and camera schemes is less than 5%	Constraint 2, Specification 2
Physical Setup Test	• Cameras track spheres from 10 feet away • Position coordinates are accurate within 5 cm from 10 feet away	Criterion 3
PTP Standard Test	• Subordinate device offset from master is less than 1 ms for 95% of samples over a 60 second interval	Standard 1
Weight Test	• The tracker weighs less than 5 oz	Criterion 2

Testing Plan

Detailed instructions for each test from Table 3 appear in this section. The passing criteria are mentioned for single execution of each respective test. For more robust results, the tests can be run multiple times. They would be deemed successful if the passing rate was 90% or higher. All programming files referenced below are found in Appendix B.

Camera Test

The camera test evaluates the effectiveness of bat tracking algorithm which records data collection rate in frames per second and displays 3D position coordinates in the terminal window. The goal of the test is to establish whether the tracking program can process at least 55 frames per second and to verify the accuracy of collected position data against physical measurements.

Materials:

- 2 Microsoft Azure Kinect cameras with power supply cords
- Trackable spherical object
- Computer or NVIDIA Jetson Orin Nano GPU
- 2 USB-C to USB-A cables
- Tape measure

Instructions:

1. Make sure to perform the test in a spot with adequate brightness and minimal glares or shadows
2. Plug in both Microsoft Azure cameras using provided power supply cords
3. Place each camera in secure spots
4. Use USB-C to USB-A cables to connect each camera to computer or GPU
5. Use tape measure to locate trackable spherical object at a position with known coordinates. Make sure that high-contrast colored spheres are visible by both cameras
6. Run *Device_Tracking_2_16.py* file in terminal
7. Observe frame rate and 3D position data in terminal
8. Slowly move trackable spherical object note changes in position data

9. Terminate *Device_Tracking_2_16.py* file (Ctrl + C)
10. Compare recorded frame rate against 55 fps threshold and 3D position against known coordinates from step 5

Pass if throughout duration of the test:

- Data collection rate exceeds 55 fps
- 3D position data is displayed in terminal
- 3D position data is within 5 cm of physically measured known location

Data Test

The data test ensures that data collection through IMU scheme can be done wirelessly and verifies the effectiveness of cross correlation program by assessing the per cent difference between aligned points obtained from camera and IMU schemes, respectively. For the latter, a standard threshold of 5% is set.

Materials:

- 2 Microsoft Azure Kinect cameras with power supply cords
- Trackable spherical object
- Computer or NVIDIA Jetson Orin Nano GPU
- 2 USB-C to USB-A cables
- USB-C to USB-C cable
- IMU transmitter and receiver
- 3.7 V Battery

Instructions:

1. Make sure to perform the test in a spot with adequate brightness and minimal glares or shadows
2. Plug in both Microsoft Azure cameras using provided power supply cords
3. Place each camera in secure spots
4. Use USB-C to USB-A cables to connect each camera to computer or GPU
5. Use USB-C to USB-C cable to connect IMU receiver to computer or GPU
6. Run *Device_Tracking_2_16.py* and *imu_data_collection.py* files at the same time
7. Terminate *Device_Tracking_2_16.py* after *imu_data_collection.py* self terminates
8. Run *angular_velo_computation.py* file
9. Run *cross_correlation.m* file (make sure to update source .csv files)
10. Check the average percent difference between aligned values (calculated automatically)

Pass: *imu_data_collection.py* runs and percent difference between aligned values from step 10 does not exceed 5%

Physical Setup Test

The physical setup test ensures that the camera scheme works at a range of 10 feet – the distance deemed as low probability of camera damage.

Materials:

- Microsoft Azure Kinect camera with power supply cord
- Trackable spherical object
- USB-C to USB-A cable
- Tape measure

Instructions:

1. Make sure to perform the test in a spot with adequate brightness and minimal glares or shadows
2. Plug in Microsoft Azure camera using provided power supply cord
3. Place camera in a secure spot
4. Stand 3 feet away from the camera and point trackable object in its direction
5. Run *single_camera_tracking.py* file
6. Slowly move away from the camera and observe changes in 3D coordinates in the terminal
7. Record distance where 3D coordinates became inconsistent with known physical location

Pass: recorded distance in step 7 is greater than 10 feet

For more robust results, the test can be run multiple times. The test is deemed successful if the passing rate is 90% or higher.

Precision Time Protocol (PTP) Standard Test

This standard test is needed to verify precise time sync between the GPU and camera using the Precision Time Protocol. By assigning one device as grandmaster clock and the other as a subordinate on the same network, the accuracy at which the sub keeps time with the master can be measured. Analysis of the offset values assesses stability or drift during operation. Ensuring that the offset remains smaller than one millisecond for 95% of trials confirms the system is sufficient for high-speed data and frame alignment.

Materials:

- Microsoft Azure Kinect Camera
- NVIDIA Jetson Orin Nano GPU
- 2 Ethernet cables
- 1 network switch with at least two ethernet ports

Instructions:

1. Connect camera and GPU to the network switch using ethernet cables
2. Configure both devices for PTP domain 0 (GPU as grandmaster and camera as subordinate)
3. In command window, run *sudo ptp4l -l <interface> -m*
4. In command window, run *sudo ptp4l -i <eth0> -m -l 5*
5. Watch offset values in real time, fluctuation normal
6. Terminate the execution command
7. Calculate the percent of offset values over a 60 second period that exceed 1 millisecond

Pass: PTP subordinate maintains an offset from grandmaster of ≤ 1 millisecond for 95% of samples over a 60-second interval

Weight Test

The weigh test will ensure that low weight criterion is met. The weight threshold of 5 oz is chosen as it typically does not have major impact on swings.

Materials:

- Scale
- Trackable spherical object
- IMU transmitter and its casing

Instructions

1. Calibrate scale according to the manufacturer instructions
2. Tare scale when empty
3. Place trackable spherical object and IMU on scale and step away (ensure nothing else is touching the scale or device)
4. Record weight

Pass: the recorded weight is less than the threshold of 5 oz.

Schematics and Drawings

This section describes how the project is built by showing general flow-charts for each program developed for the project.

Figure 8 illustrates the framework for the dual camera tracking pipeline. The program starts by initializing PyKinect and a Zero Message Queene (ZMQ) publisher. then attempts to open the first camera indexed as Camera 0. If the initialization fails, the program is closed as there are no active cameras. After the successful initialization of Camera 0, attempt to access Camera 1 is performed. If unsuccessful, the program falls back to single camera tracking mode. When both cameras are launched and are interlaced by the time hard-coded offset of 16.67 millisecond, the capture thread is triggered. The main loop polls the latest color and depth frame and validates it. Invalid frames trigger a retry, while valid frames are downscaled and HSV-converted by a factor of 3 to speed up processing and reduce bandwidth. For a mask of each color, if a blob passes the minimum area and region of interest (ROI) gate, the system samples depth and projects the pixels to 3D. The result is then published over ZMQ updating the on-screen overlay in real time. If no blob qualifies, that color's last position is cleared. The loop continues frame-by-frame until the user ends the program by pressing Q or ESC at which point the tracking session terminates.

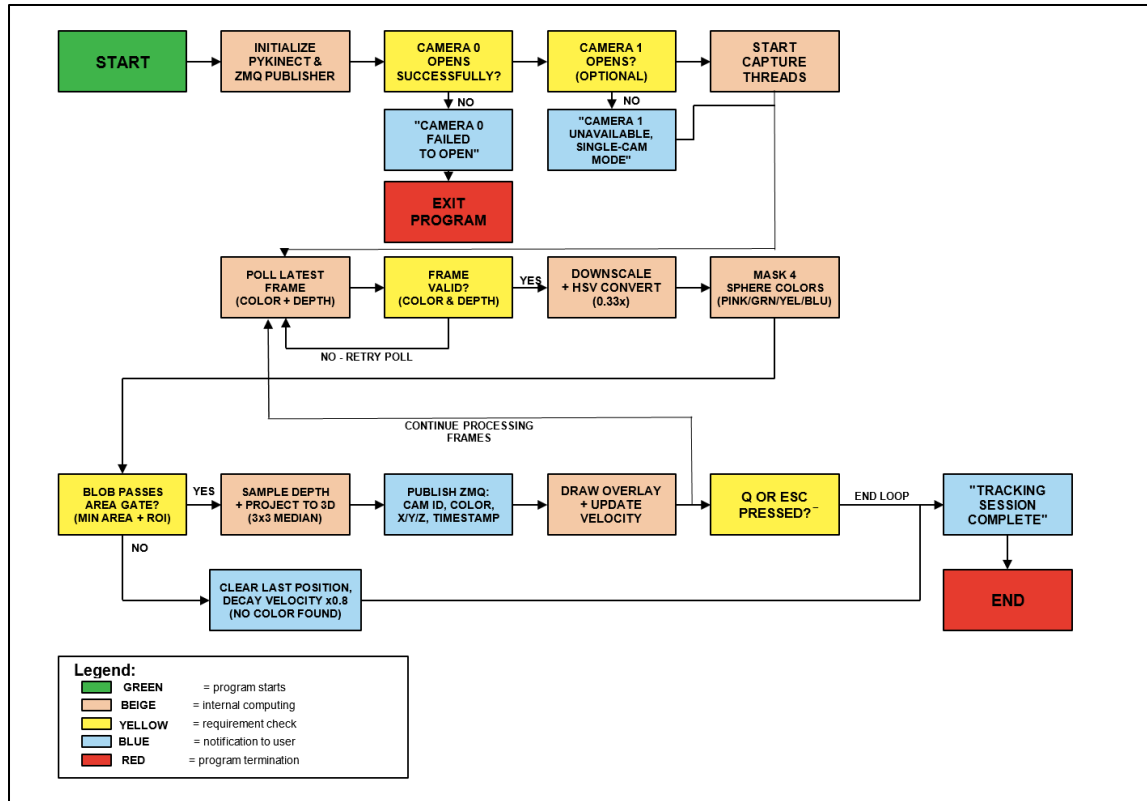


Fig 8. Camera Tracking Program Flow Chart

Next, Figure 9 contains the flow chart for IMU data collection. Few global variables are initialized at the top of the program including desired number of data points, buffer size, and data storage arrays. Then, the program checks whether receiver is connected to the communication port of either GPU or computer. The program is terminated if the connection is missing. Otherwise, a serial publisher is initialized. Next, the program makes sure that the port has not been corrupted. If so, the user is instructed to take a swing. As soon as that message appears, the data collection loop begins until the desired number of data points is reached. At that point, "Data collection is complete" message is displayed and the program self terminates.

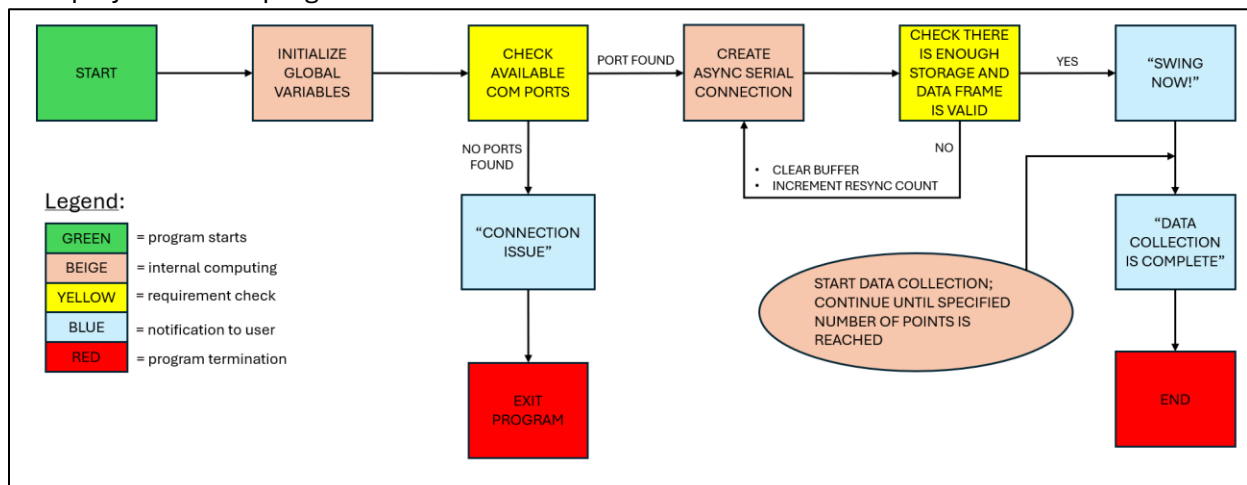


Fig. 9. IMU Data Collection Program Flow Chart

Figure 10 shows the flow chart of the program that computes 6 degree-of-freedom (DOF) pose estimation from the trackable spherical object. First, a ZMQ connection is established to the GPU streaming live position data. The program then waits to receive a payload and checks whether at least three of the four colored markers are visible. If not, the frame is skipped and the program returns to waiting. Otherwise, a shared origin anchor is set or applied using the first frame from camera 0 to normalize coordinates across cameras. The matched point sets B and W are then constructed from the known model coordinates and the observed world coordinates respectively. Horn's method, implemented via the Davenport q-method, is then applied by solving for the largest eigenvector of a 4×4 symmetric cross-covariance matrix, yielding the optimal quaternion q , rotation matrix R , and translation vector t describing the pose of the object. The per-point reprojection errors and RMS error are then computed to evaluate solution accuracy. Finally, all outputs are assembled into a record and written to a JSONL log file, after which the program loops back to await the next payload.

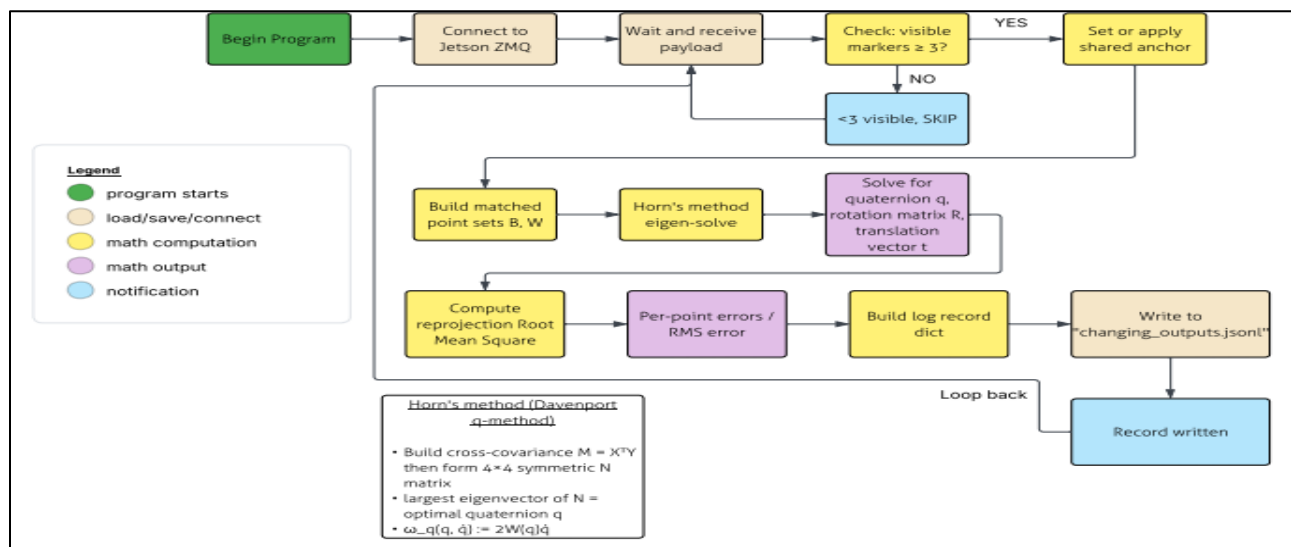


Fig 10. Quaternion Calculation Program Flow Chart

Figure 11 presents the flow chart of the program that computes angular velocity values from quaternion data calculated as described above. First, timestamps and quaternion coordinates are extracted from JSON objects loaded into the program. Then, discrete quaternion data is turned into continuous function using imported Piecewise Cubic Hermite Interpolating Polynomial (PHCIP) interpolator – $q(t)$. Then, the program takes a time derivative of $q(t)$ resulting in $\dot{q}(t)$. These values are necessary to compute a W matrix and angular velocity as shown at the bottom of the figure [6]. Once angular velocity is computed, it is saved to .csv file and the user is notified with a corresponding message.

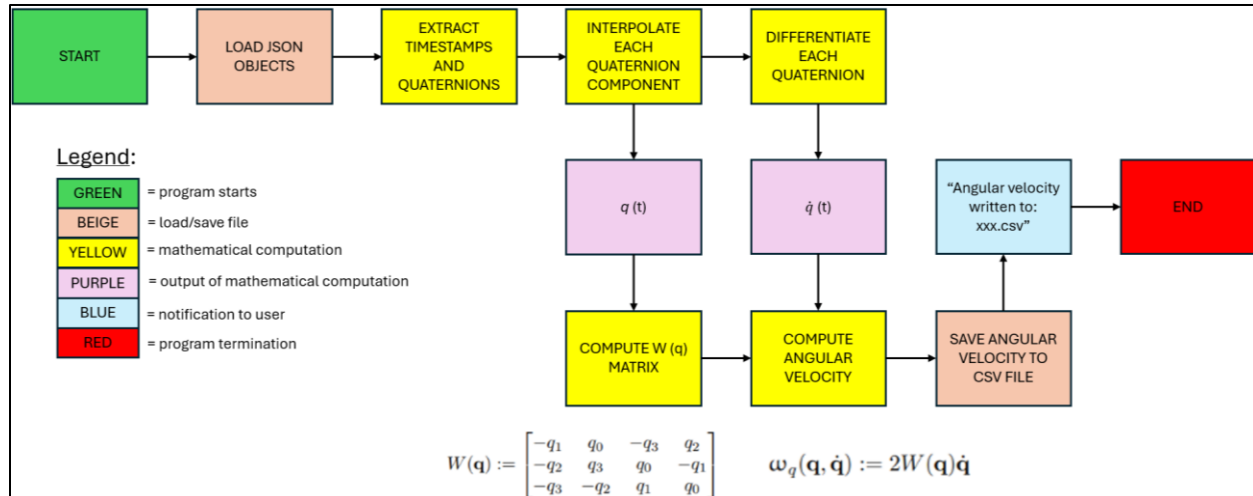


Fig. 11. Angular Velocity Computation Program Flow Chart

Finally, Figure 12 shows the flow chart for the cross-correlation program that aligns data collected from camera and IMU schemes. First, files containing respective data are loaded into the program. Then, time and three components (x-, y-, and z-) of angular velocity are extracted. Angular velocity magnitude is computed from the extracted values. Then, lagging time of the camera stream is estimated using the cross-correlation algorithm. The time scale of the camera stream is then shifted by the corresponding delay and the percent difference between newly aligned values is computed.

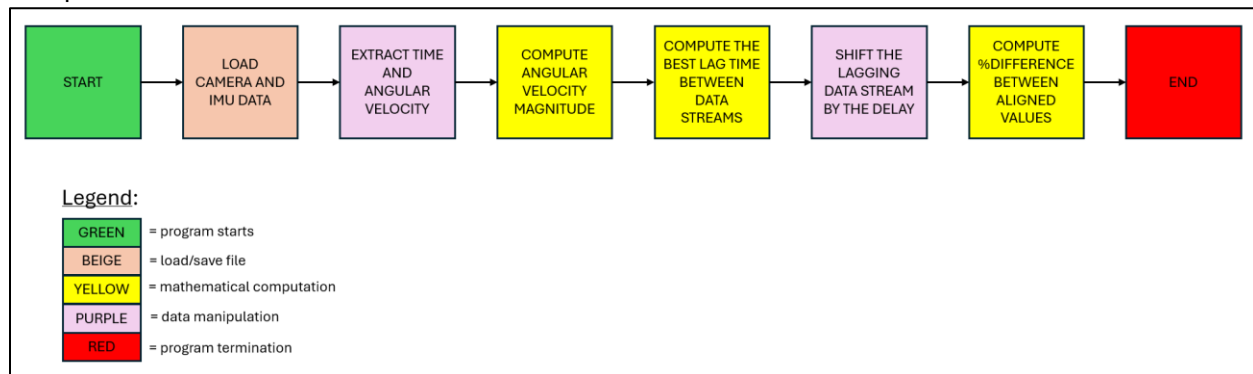


Fig.12. Cross-Correlation Program Flow Chart

Client Materials

As can be inferred from previous sections, this project has many moving parts. Hence, the goal of this section is to summarize instructions for how to run the project as well as what the next steps should be in its development.

In general, the project can be split up into 6 major tasks. Each task, except for the last, has a unique program to execute it given the hardware and source data requirements are met. Table 4 summarizes what each task is, what program executes it, and the requirements to run it.

Table 4. Project Task Summary

Task (#)	Hardware/Source Data Requirements	Program
Collect camera data (1)	Microsoft Azure Cameras (2x) Trackable spherical object	<i>Device_Tracking_2_16.py</i>
Collect IMU data (2)	IMU transmitter and receiver	<i>imu_data_collection.py</i>
Calculate quaternion coordinates (3)	3D Position data	<i>Quaternion_Scheme_2_12.py</i>
Calculate angular velocity (4)	Quaternion coordinates	<i>angular_velo_computation.py</i>
Align camera and IMU data (5)	Camera angular velocity IMU angular velocity	<i>cross_correlation.m</i>
Produce swing rendering (6)	Headwear with Kalman filter Aligned 3D position data IMU data	NOT YET DEVELOPED

Importantly, the tasks listed in the table should be run sequentially as some of them produce data that needs to be used as an input for the next program. For example, 3D-position data comes from Task (1), and is used as an input to Task (3). Analogously, quaternion coordinates from Task (3) are used to calculate camera angular velocity in Task (4), which is then supplied into Task (5) along with IMU data from Task(2). In the final task, IMU (Task (2)) and aligned 3D-position data (Task (5)) would be uploaded into a VR headset with embedded Kalman filter to produce a swing rendering.

Following are brief instructions for setting up each developed task described in Table 4 as well as expected results.

Task (1): Camera Data Collection

1. Set up two Microsoft Azure cameras as shown in Figure 2 (7 feet away from the tracker)
2. Plug in both cameras using provided power supply cords
3. Run *Device_Tracking_2_16.py* program as long as necessary
4. 3D position coordinates are continuously displayed in the terminal and are stored to database to specified file directory

Task (2): IMU Data Collection

1. Charge 3.7V battery and plug it into wireless IMU transmitter
2. Insert transmitter into specifically designed casing shown in Figure 4
3. Make sure IMU receiver is plugged into computer or GPU for data transfer
4. Run *imu_data_collection.py*

5. Take a swing when instructed
6. Data is stored in 'reports' folder of the specified file directory. Make sure to create the folder before running the program!

Task (3): Quaternion Coordinate Calculation

1. Ensure GPU is powered on and connected to the same network as the host computer
2. Verify the GPU IP address matches the one specified at the top of the program (JETSON_IP = "10.132.52.95")
3. Run *Device_Tracking_2_16.py*
4. Run *Quaternion_Scheme_12_2.py*
5. At least three markers must always be visible to the cameras for a valid pose solution
6. Quaternion coordinates, rotation matrix, translation vector, and reprojection errors are continuously computed and stored to *changing_outputs.jsonl* in the same file directory as the program

Task (4): Angular Velocity Calculation

1. Make sure to load the right .jsonl file and that it is saved in the same location as *angular_velo_computation.py*
2. Run *angular_velo_computation.py*
3. Data is stored in .csv format in the same file directory

Task (5): Cross-correlation

1. Make sure to load the right camera and IMU data by specifying their file names at the top of *cross_correlation.m* file
2. Make sure to install Signal Processing Toolbox before running the program
3. Run *cross_correlation.m*
4. Output of the program is aligned camera and IMU data as well as average percent error between aligned values
5. Output data is not automatically stored because the format required by Kalman filter has not been explored

Moving forward, the priority should be to produce swing rendering by formatting data from the first 5 tasks to adhere to the requirements of the Kalman filter. However, each individual task can also be enhanced with more efficient programming as discussed below in Chapter 6.

Chapter 4: Broader Impacts

Public Health, Safety, and Welfare

The project provides a scheme that allows baseball and softball players optimize their training processes including the time and effort they have to allocate to that aspect of their routine. Therefore, the product would be very beneficial from the athlete health perspective. That is, the athletes would be able to dedicate more time to rest and recovery which would minimize the risk of

injury during practice and games as a byproduct of players having more time to put their bodies in optimal physical condition.

Global, Cultural, and Social

Baseball is one of the most popular sports in the United States. The sport of baseball is not only an integral part of many young children growing up in the country but is also an important aspect of socializing for people in all age groups. Regardless of what the connection is to the sport, all baseball fans like better quality of the game to fulfil their playing or watching experience. Since the product provides a low-cost way to improve batting performance, the project will contribute to the growth of baseball on the global scale indirectly impacting cultural and social standards related to the game.

Environmental

The project is environmentally friendly as it uses high-efficiency battery for its tracking scheme and requires minimal power supply which can be supported by most households and industrial facilities without detrimental impact to the environment.

Economic

This project follows a strict budget given the fixed items involved in the setup. The NVIDIA Jetson Orin Nano and the Azure Kinect Cameras are dictating financial factors. Externally, this setup could save people great time and money that would otherwise be spent on coaching and possibly a less effective practice routine. Allowing users to view their personal metrics can allow for a one-time investment for effective return both physically and financially.

Engineering Standards

Engineering standards followed during the design process addressed the health and safety concerns of the users, which is crucially important for any engineering project as the customers must feel confident and comfortable using the product. Specifically for this project, the product was designed in a way that the users do not need to spend an extended amount of time with VR/XR technology ensuring they do not feel overwhelmed. Additionally, there were standards about using the specific tracking mechanisms like cameras and IMU that provided guidance in programming and assessing the effectiveness of those devices.

Risk Assessment

As has been established in the literature review, the project provides strong benefits for the users and given that the team followed engineering standards that were concerned with health and safety of athletes, there is very little risk associated with the product. Of course, just like in any other athletic activity, the players are not immune from getting injured if used without proper form or preparation. However, there are no direct implementation risks from the design standpoint.

Chapter 5: Project Management

Schedule

The Gantt chart, shown in Figure 13, outlines both the planned and actual schedules for the project, spanning 17 weeks from August 17, 2025 to December 14, 2025.

The most significant source of delay throughout the project stemmed from hardware limitations encountered when attempting to run the Microsoft Azure Kinect depth sensing library. The GPU lacked sufficient bandwidth and processing power to handle the computational demands of the library, which caused substantial setbacks in the object tracking program and propagated delays throughout the remainder of the pipeline. A significant amount of time that had been allocated to software development was instead spent troubleshooting compatibility issues, identifying the hardware bottleneck, and ultimately finding workarounds to extract usable camera data. This caused the object tracking and IMU data collection programs to exceed their planned durations considerably. As a downstream consequence, the database integration, cross-correlation program, and Kalman filter tasks all suffered. The cross-correlation program reached only 25% completion and the Kalman filter swing rendering task was never started. Had the GPU limitation been identified earlier in the project timeline, alternative hardware could have been sourced sooner, and the later processing tasks would have had adequate time for full implementation.

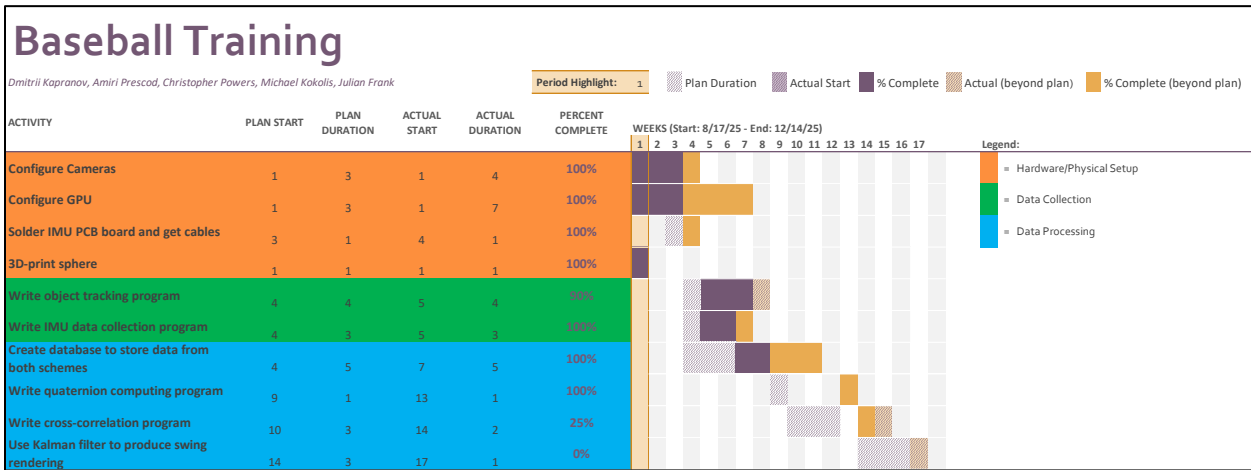


Fig. 8. Enhanced Baseball Training Gantt Chart

Personnel

Dmitrii Kapranov – As a student with electrical engineering major and minor in mathematics and mechatronics, Dmitrii was well equipped to work on every subsystem in the project. His electrical engineering background helped him configure cameras while mathematics allowed him to excel at data processing work. Additionally, he was able to leverage his mechatronics related knowledge to theorize optimal shape for the 3D-printed object. Due to his versatility in all aspects of the project, Dmitrii was also designated team leader responsible for task delegation. Overall, Dmitrii has averaged around 6 hours of project work per week.

Chris Powers— With a strong background in programming, Chris plays a key role in interfacing code with the Kinect camera. His previous experience with Python and the relevant libraries, gained through past coding projects, has been instrumental in developing the bat tracker using Azure Kinect Cameras. Chris has worked on this project for around 3 hours per week. As a member of the club baseball team, Chris also brings a firsthand understanding of the sport, allowing him to provide critical insights into the baseball-specific aspects of our project.

Julian Frank – As the team’s only computer engineer, Julian brings extensive software expertise. His academic exposure to multiple programming languages allows the team to test different languages and optimize the approach to the selected solution. Additionally, his experience as a club baseball player enables him to bridge the gap between technical solutions and real-world athletic application, offering unique insights that others might overlook. Julian has dedicated 3 hours per week to this project during its duration.

Michael Kokolis – Specializing in hardware, Michael is particularly adept at working with the NVIDIA GPU, which will be crucial for processing IMU data and running the programming algorithms efficiently. His inquisitive nature ensures that he consistently asks the right questions at the right time, helping the team identify potential roadblocks early and accelerating overall progress. Additionally, he is able to work alongside Chris in integrating the cameras with the GPU for processing. Michael has worked on this for an average of 3 hours a week.

Amiri Prescod – A versatile and well-rounded team member, Amiri possesses strong coding skills—especially in Python—through his computer science minor. Additionally, he has experience working with hardware, including the GPU, making him an asset in both software and hardware domains. Amiri has worked on his contributions for around 3 hours per week for the project’s duration.

Parts and Components

Overall, there were two major components necessary to execute the project.

Graphics Processing Unit:

- Model: Jetson Orin Nano
- Manufacturer: NVIDIA
- Headquarters: Santa Clara, CA
- Use: Processing positional data from the two Azure Kinect cameras, motion data from the IMU, and merging the data to produce a rendering of the swing

Cameras:

- Model: Azure Kinect
- Manufacturer: Microsoft
- Headquarters: Redmond, WA
- Use: Gathering positional data via 3D printed sphere object and OpenCV library to merge with IMU data in the GPU

Skills and Tools

Identify what techniques, skills, and tools you used in your project. Types of prototypes, software packages, instruments for testing, etc, should all be included.

Skills:

- 3D Design – creating tracking object and IMU enclosure scheme
- Python – language used for running camera and IMU schemes
- OpenCV – Python library used for camera tracking
- Linux – OS used on the GPU
- Digital Scale – measuring total weight of the scheme

Budget

Table 5 demonstrates the total cost that the team committed to realize the project. Luckily, all the components were either owned by project advisor Dr. Jupina, accessible to students through university allowance, or open-source technology that can be used at no charge. Hence, the total budget for the team came out to a total of \$0.

Item	Manufacturer	Cost per unit	Num. Units	Total Cost
Microsoft Azure Kinect Camera	Microsoft	Free (Advisor already owned)	2	0
NVIDIA Jetson GPU	NVIDIA	Free (Advisor already owned)	1	0
Inertial Measurement Unit	Custom	Free (Advisory already owned parts)	1	0
Python and Libraries	Open Source	Free	5	0
Microsoft Visual Studio	Microsoft	Free	5	0
Hand Tracking Device	Custom	Free (Villanova 3D print)	1	0
Monitor, Keyboard, Mouse	Samsung	Free (Team previously owned)	1	0
Final Total				\$0

The cost of replicating the project in the industry is also not off the charts. The major expenses come from Microsoft Azure cameras and NVIDIA Jetson GPU that are estimated to cost around \$400 and \$250 per device respectively. The IMU microprocessor is a standard low-cost device that can be obtained for around \$50. Additionally, there are other miscellaneous expenses including 3D-print filament, computer monitor, and labor to install and configure utilized hardware. The total cost estimation for the project came out to a reasonable \$1,355 as broken down below:

- NVIDIA Jetson Orin Nano GPU - \$250

- Microsoft Azure Kinect Cameras (2) - \$800
- IMU - \$50
- 3D Printed Trackable Spherical Object - \$5
- Keyboard, Mouse, Monitor - ~150
- Labor - \$100

One of the initial design goals of the project was to make the product affordable to implement. The Budget and industry cost analysis confirms that this objective has been achieved.

Chapter 6: Achievements and Conclusions

Achievements

GPU + Camera Scheme

Going into the project, there were several options as to how the cameras should be connected to the GPU so that data could be processed quickly and efficiently. The GPU initialization itself took longer than expected (see Schedule section of Chapter 5). So, running test camera codes were delayed. During technical implementation stage of the project, several camera tracking codes ran successfully on the GPU. Some programs accurately tracked each color of the spherical object on and displayed respective location of each color in the terminal window. Instead of desired 10 feet as outlined in physical setup test in Chapter 3, data collection became inconsistent at around 7 feet leading the team to conclude that extra safety mechanisms such as L-screen or shielding boxes are necessary to fully implement the product. Just as expected from ethnographic research findings in Chapter 2, lighting played a major factor in effectiveness of object tracking program operation. That is because paint is susceptible to reflection from overhead lighting. Interlacing cameras to double the frame rate and normalizing them to a single point was another major accomplishment as it provided the ability to collect almost twice as much data compared to a single camera scheme. Finally, being able to develop a fast and accurate ZMQ method for transferring data from GPU to computer was another success especially after considering other possibilities like USB and Bluetooth.

IMU Scheme

The ability to collect data using IMU tracker was a big success. Since all on-chip sensors came originally pre-configured, the major challenge of this scheme was to organize and format file structure, so the program runs smoothly and resulting data is easy to utilize for further computations. This was successfully executed by creating a folder that records a timestamp of the recorded data as well as adding respective headers to each recorded value in the spreadsheet. This organization made IMU data easy to use in cross-correlation procedure as discussed in Chapter 3. Additionally, it was crucial that the IMU transmitter meets a low weight criterion from Chapter 3 not to impede the quality of taken swings. The goal was successfully implemented with a Velcro based fastener.

Swing rendering

Unfortunately, the team did not get to use Kalman filter to produce a virtual swing rendering. However, most necessary building blocks have been put in place to begin working toward it. At this point, the product obtains all necessary data to run the Kalman filter under perfect conditions. Hence, at this point, the challenges of getting the project over the finish line consist of making the product more adaptable to physical environments and accumulating data to match requirements of the Kalman filter. The team views its effort in data collection and processing to the point where virtual rendering is within reach as another major accomplishment.

Conclusions

Throughout the course of the project, there were successful runs of tracking codes, tests with the cameras and GPU, IMU data captures, and data transfers. Though still in a preliminary development stage, the foundation of the overall goal was set forth in establishing how each scheme should operate and how they will combine to create the full swing rendering.

The color tracking scheme is sensitive to lighting conditions and limited by camera resolution. Therefore, it is recommended to consider alternative tracking algorithms that would maintain high degree of accuracy in uncontrolled physical conditions. Additionally, the 3D printed tracking object could be altered so that lighting has a lesser effect on the performance of the tracking codes. The framerate of the cameras significantly hinders the overall performance of the scheme. Should other options become available, it is recommended to use a different camera with depth-sensing capabilities with a higher framerate.

IMU tracking scheme is currently designed to terminate when a specified number of points have been collected, which puts pressure on the batter to swing within that certain time frame. This could present limitations when interacting with a live at-bat situation. In the future, the scheme should be able to automatically detect when the batter is swinging, and capture data from a certain time before said moment and a certain time after the swing is over and the IMU values have started to indicate a finished swing.

As previously stated, the swing rendering did not get put into practice due to unforeseen delays in the project. The use of a Kalman filter should use cross-correlated positional data from the cameras and with motion data from the IMU to smoothly render the swing in a 3D space. Once the rendering is displayed, the focus should shift to gathering metrics from the rendering such as moment of highest velocity, moment of impact, and angle of attack. These metrics should all be able to be gathered from the existing data from the combination of the cameras and IMU, provided that the suggestions and recommendations listed above were explored.

References

- [1] 3079-2020 - *Ieee Standard For Head-mounted Display Sickness Reduction Technology*. IEEE, 2021.
- [2] J. Yao and S. Warren, "Applying the ISO/IEEE 11073 standards to wearable home health monitoring systems," *J. Clin. Monit. Comput.*, vol. 19, no. 6, pp. 427–436, Dec. 2005, doi: 10.1007/s10877-005-2033-7.
- [3] Z. Liyuan, "Evaluation of a Virtual Reality-based Baseball Batting Training System," 2019.
- [4] Minal Goyal, Manharleen Kaur, Loveleen, and Navneet Kaur, "Effect of Immersive Virtual Reality Environment on Reaction Time, Agility and Coordination among Collegiate Students," 2024.

- [5] Committee on Sensor Technology (TC-9), "IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems IEEE Instrumentation and Measurement Society," *IEEE Instrumentation and Measurement Society*, Jul. 2008.
- [6] J. Diebel, "Representing Attitude: Euler Angles, Unit Quaternions, and Rotation Vectors," 2006.
- [7] J. Park, J.-H. Kim, and S. H. Kim, "The Initiative Design to Combine Real Bats and Balls with Online Baseball Contents," 2011.
- [8] Balvinder. Shukla, S. Kumar. Khatri, and P. K. . Kapur, *Moving Object Tracking Using Optical Flow and Motion Vector Estimation*. IEEE, 2015.
- [9] Z. Dou and F. Wang, "Application of Reinforcement Learning Algorithms in Intelligent Sports Training Video Tracking," in *Proceedings of 5th International Conference on IoT Based Control Networks and Intelligent Systems, ICICNIS 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 948–953. doi: 10.1109/ICICNIS64247.2024.10823338.
- [10] M. Majcher and B. Kwolek, "Object pose tracking using multimodal knowledge from RGB images and quaternion-based rotation contexts," *Appl. Soft Comput.*, vol. 170, Feb. 2025, doi: 10.1016/j.asoc.2025.112699.
- [11] A. C. Sankaranarayanan, A. Veeraraghavan, and R. Chellappa, "Object detection, tracking and recognition for multiple smart cameras," *Proceedings of the IEEE*, vol. 96, no. 10, pp. 1606–1624, 2008, doi: 10.1109/JPROC.2008.928758.
- [12] K. Ryu, H. Kim, and S. Lee, "A deep learning model based on sequential object feature accumulation for sport activity recognition," *Multimed. Tools Appl.*, vol. 82, no. 24, pp. 37387–37406, Oct. 2023, doi: 10.1007/s11042-023-15022-8.
- [13] Q. Wang and C. Zhao, "Sports Video Object Tracking Algorithm Based on Optimized Particle Filter," *EAI Endorsed Transactions on Scalable Information Systems*, vol. 11, no. 3, pp. 1–8, 2024, doi: 10.4108/eetsis.3935.
- [14] A. Naresh, B. N. Kumar, and V. Sravanthi, "IoT Based Object Detection and Identification with Open CV using Web CAM," in *Proceedings - 2024 1st International Conference on Innovative Sustainable Technologies for Energy, Mechatronics and Smart Systems, ISTEMS 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/ISTEMS60181.2024.10560113.
- [15] Kiyoshi. Kiyokawa, "What Can VR Systems Tell Sports Players? Reaction-Based Analysis of Baseball Batters in Virtual and Real Worlds," 2018.
- [16] J. S. Putranto, J. Heriyanto, Kenny, S. Achmad, and A. Kurniawan, "Implementation of virtual reality technology for sports education and training: Systematic literature review," in *Procedia Computer Science*, Elsevier B.V., 2022, pp. 293–300. doi: 10.1016/j.procs.2022.12.139.

- [17] E. Stewart, M. Stewart, J. Simpson, A. Knight, H. Chander, and R. Shapiro, "Sequential order of swing phase initiation in baseball," *Journal of Sports Analytics*, vol. 6, no. 3, pp. 199–204, Sep. 2020, doi: 10.3233/jsa-200394.
- [18] G. Healey, "The New Moneyball: How Ballpark Sensors Are Changing Baseball," Nov. 01, 2017, *Institute of Electrical and Electronics Engineers Inc.* doi: 10.1109/JPROC.2017.2756740.
- [19] D. Oscar. Suarez's and S. Jesus. Tercero, *Learning Image Processing with Opencv*. Packt Publishing, 2015.
- [20] S. Valladares, M. Toscano, R. Tufiño, P. Morillo, and D. Vallejo-Huanga, "Performance Evaluation of the Nvidia Jetson Nano Through a Real-Time Machine Learning Application," 2021, pp. 343–349. doi: 10.1007/978-3-030-68017-6_51.
- [21] 2888.5-2024 - *IEEE Standard for Virtual Training System Evaluation Methods*. IEEE, 2024.
- [22] 1589-2020 - *IEEE Standard for Augmented Reality Learning Experience Model*. IEEE, 2020.
- [23] Horn, B. K. P. (1987). *Closed-form solution of absolute orientation using unit quaternions*. *Journal of the Optical Society of America A*, 4(4), 629–642.

Appendix A: Ethnographic Research

Motion/Object Tracking:

- [7] Park et al. (2011)
 - Camera-based object tracking is possible
 - Accurate data at 2000 frames per second
 - VR can be used to visualize real life effects
- [8] Kale et al. (2015)
 - Better accuracy from optical flow and motion vector estimation
 - Improved performance with pre-recorded videos
- [9] Dou & Wang (2024)
 - Modern tracking algorithms that utilize vector estimation achieve extremely high accuracy (>99%)
 - Algorithms tested in sport settings including college basketball, professional baseball, swimming, and soccer
- [10] Majcher & Kwolek (2025)
 - X0Net improves robustness in 3D pose tracking with added rotation data
 - Joint keypoint detection improves object tracking
- [11] Sankaranarayanan et al. (2008)
 - Camera schemes struggle from inconsistent illumination
 - Geometric challenges of multi-camera schemes are solved with projective geometry, epipolar constraints, and homography
 - Moving objects are detected using background subtraction techniques
- [12] Ryu et al. (2023)
 - Tracking improves when analyzing motion over time, not just single frames
 - Neural networks can recognize complex activities (like swings) from video.
 - Proposed splits activity into activity several features which is used as training data set
- [13] Ma (2016)
 - Real-time algorithm to track athlete movement
 - Use of cross-correlation to improve signal quality
- [14] Naresh et al. (2024)
 - OpenCV is better at performing mundane tasks in object tracking
 - Raspberry Pi 3 is not suitable for object tracking due to limited processing capabilities

Virtual Reality vs. Traditional Training:

- [3] Zou et al. (2019)
 - VR technology showed improvement in players' performance
 - VR technology used for batting practice can allow players to get instantaneous access to critical information
- [15] Isogawa et al. (2020)
 - Players react slightly slower in VR than in real life
 - VR can be effective, but should closely match real conditions
- [16] Putranto et al. (2023)
 - VR helps improve decision-making and technique in sports
 - VR headwear limits range of motion

[4] Goyal et al. (2024)

- VR training can improve reaction time, coordination, and agility
- Controlled study concludes that performance gains are comparable to real-world training

[17] Stewart et al. (2020)

- A good swing follows a specific sequence: lower body → upper body → hands
- Skilled players have more consistent timing than beginners
- Timing of movement phases directly affects performance

Hardware & Tools:

[18] Healey (2017)

- High-end tracking systems use cameras for tracking
- Wide range of data can be computed with cameras alone

[19] Garcia et al. (2015)

- Processing time determines practicality of the project
- OpenCL in OpenCV is set for measuring acceleration
- Color-based tracking requires splitting of RGB layers

[20] Tufino et al. (2021)

- Reduced reliance on cloud computing using on-device AI
- On-device AI is effective in all lighting conditions
- Jetson Nano optimizes resource utilization using integrated machine learning systems

IEEE Standards:

[21] IEEE VR Evaluation Standard (2024)

- A good VR system should be tested for effectiveness, usability, and user acceptance
- Questionnaires are perfect for compiling insight

[1] IEEE HMD VR Standard (2021)

- Defines requirements to reduce motion sickness in VR systems
- Covers visual quality, sound, and physical design

[2] Yao & Warren (2006)

- Adjustments to original designs that do not meet all IEEE standards are a must
- Wearable technology can be made safe with proper implementation
- Communication, power, and synchronization are key challenges

[22] Wild et al. (2020)

- ARLEM defines data formats focusing on interoperability and reusability across different platforms
- DevOpsUseXR integrates AR into software development

Notes on Interview with Mr. Edward Dougherty:

Interview Information:

- Name: Edward Dougherty
- Location: Campus Corner
- Date and Time: 3/13/2025 at 11.30 AM
- Duration: 1 hour

General questions:

1. What kind of object tracking cameras have you worked with and what was your experience like?
2. What are the common challenges when writing object tracking programs?
3. What are some intermediate goals that need to be accomplished for this application? What is the most efficient way to divide up the work?

Notes:

- Background: Mr. Dougherty has worked on a number of object-tracking projects throughout his professional career and specialized in camera selection and writing programs for effective data collection. He started working on a baseball application a few years ago with his friend Dan Hennigan.
- Motivation: to help players with bat selection based on their swing and physical characteristics
 - o Used many different metrics from recordings and traditional data → will send specific metrics later
- Accuracy of the measured data depends on how zoomed the recording is and speed of camera (sampling rate)
 - o At least two points need to be recorded to figure out the trajectory of the object and information about it
 - o Crossing of the wrists may be a challenge in baseball application
 - o Suggests to put one camera in front and one camera directly above the batter to map the path of the swing
- Common challenges of working with object-tracking programs are lighting, camera speed, and resolution
 - o Lighting issues can be dealt with if you add an easily recognizable feature on the object of interest such as a blinking LED
 - o Alternatively, one might consider controlling the lighting conditions
- Any programming language is compatible for application in question → Python and C are recommended
- Important to not overload the operating system → need fast processing unit
- Design process steps
 - o List the rules for the project → what is feasible and what are the minimum goals
 - o Determine the audience → what level baseball players
 - o Given the equipment, work backwards to find optimal solutions and limitations
 - o Determine the technical parameters to meet
 - o Determine hardware setup
- Project specific recommendations
 - o Assign one parameter per student to write a program for OR
 - o Split by areas of “expertise”
 - Programming
 - Setup
 - Writing report
- Important to set feasible milestones and achieve them in timely fashion
- Specific plan for presentation in the fall

Appendix B: Code/Schematics/Etc.

Any raw code or schematics that are too big include in chapter 4, but are necessary to turn over to your client. You may need an appendix C for more of these materials, project dependent.

Camera Tracking: [Device_Tracking_2_16.py](#)

```
import cv2 as cv
import numpy as np
import math
import time
import threading
from collections import deque
import queue
import zmq
import json

# ----- Azure Kinect -----
import pykinect_azure as pykinect
from pykinect_azure import (
    K4A_CALIBRATION_TYPE_COLOR,
    K4A_CALIBRATION_TYPE_DEPTH,
    k4a_float2_t,
)

cv.setUseOptimized(True)
cv.setNumThreads(16)

# ===== CONFIG =====
COLOR_RES = pykinect.K4A_COLOR_RESOLUTION_720P
DEPTH_MODE = pykinect.K4A_DEPTH_MODE_NFOV_2X2BINNED

HAVE_SYNC_CABLE = False
SUBORD_DELAY_US = 16667

# Detection tuning
FAR_MODE = True
USE_ROI_TRACK = True
ROI_RADIUS = 120

DETECTION_SCALE = 0.33

H_TOL = 10
S_MIN, S_MAX = 150, 255
V_MIN, V_MAX = 120, 255
```

```

FAR_MIN_AREA_FRAC = 2e-5
MIN_AREA_ABS_FLOOR = 8.0

FONT = cv.FONT_HERSHEY_SIMPLEX

NEON_HUES = {
    "neon_pink": [165, 175, 5, 15],
    "neon_green": [55, 65, 75],
    "neon_yellow": [28, 25, 30],
    "sky_blue": [105, 95, 115],
}

DRAW_COLORS = {
    "neon_pink": (255, 0, 255),
    "neon_green": (57, 255, 20),
    "neon_yellow": (0, 255, 255),
    "sky_blue": (255, 191, 0),
}

KERNEL_SIZE = (3, 3) if FAR_MODE else (5, 5)
KERNEL = cv.getStructuringElement(cv.MORPH_ELLIPSE, KERNEL_SIZE)

# ===== ZMQ PUBLISHER =====
class CoordinatePublisher:
    def __init__(self, port=5555):
        self.context = zmq.Context()
        self.socket = self.context.socket(zmq.PUB)
        self.socket.bind(f"tcp://*:{port}")
        print(f"Publishing coordinates on port {port}")
        time.sleep(0.5) # Allow time for socket binding

    def publish(self, camera_id, color_name, x_m, y_m, z_m, timestamp):
        """Publish 3D coordinates with camera ID and timestamp"""
        message = {
            "camera_id": camera_id,
            "color": color_name,
            "position": {
                "x": x_m,
                "y": y_m,
                "z": z_m
            },
            "timestamp": timestamp
        }
        self.socket.send_string(json.dumps(message))

```

```

def close(self):
    self.socket.close()
    self.context.term()

# ===== HELPERS =====
class FPSMeter:
    def __init__(self, window=30):
        self.t_last = time.perf_counter()
        self.samples = deque(maxlen=window)
    def tick(self):
        t = time.perf_counter()
        self.samples.append(t - self.t_last)
        self.t_last = t
    def fps(self):
        if len(self.samples) < 2:
            return 0.0
        avg = sum(self.samples) / len(self.samples)
        return (1.0 / avg) if avg > 0 else 0.0

def build_ranges_for_hue(h_center, h_tol, s_min, s_max, v_min, v_max):
    lo_h = h_center - h_tol
    hi_h = h_center + h_tol
    if lo_h < 0:
        return [
            (np.array([0, s_min, v_min]), np.array([hi_h, s_max, v_max])),
            (np.array([180 + lo_h, s_min, v_min]), np.array([179, s_max, v_max])),
        ]
    elif hi_h > 179:
        return [
            (np.array([lo_h, s_min, v_min]), np.array([179, s_max, v_max])),
            (np.array([0, s_min, v_min]), np.array([hi_h - 180, s_max, v_max])),
        ]
    else:
        return [(np.array([lo_h, s_min, v_min]), np.array([hi_h, s_max, v_max]))]

def make_color_ranges():
    ranges = {}
    for name, centers in NEON_HUES.items():
        spans = []
        for c in centers:
            spans.extend(build_ranges_for_hue(c, H_TOL, S_MIN, S_MAX, V_MIN, V_MAX))
        ranges[name] = spans
    return ranges

COLOR_RANGES = make_color_ranges()

```

```

def clamp_roi(x, y, r, W, H):
    x0 = max(0, x - r); y0 = max(0, y - r)
    x1 = min(W, x + r); y1 = min(H, y + r)
    return x0, y0, x1, y1

def choose_best_blob(blobs, pred):
    if not blobs:
        return None
    if pred is None:
        return max(blobs, key=lambda b: b["area"])
    px, py = pred
    return min(blobs, key=lambda b: (b["center"][0]-px)**2 + (b["center"][1]-py)**2)

def safe_sample_depth(depth_img, x, y, win=3):
    H, W = depth_img.shape[:2]
    if x < 0 or y < 0 or x >= W or y >= H:
        return 0
    d = int(depth_img[y, x])
    if d > 0:
        return d
    x0, y0 = max(0, x - win), max(0, y - win)
    x1, y1 = min(W, x + win + 1), min(H, y + win + 1)
    patch = depth_img[y0:y1, x0:x1]
    vals = patch[patch > 0]
    if vals.size == 0:
        return 0
    return int(np.median(vals))

def pixel_to_3d(device, px, py, depth_mm):
    if depth_mm <= 0:
        return None
    try:
        pixels = k4a_float2_t((float(px), float(py)))
        pos3d = device.calibration.convert_2d_to_3d(
            pixels, depth_mm, K4A_CALIBRATION_TYPE_COLOR, K4A_CALIBRATION_TYPE_COLOR
        )
        if pos3d is None:
            return None
        return (pos3d.xyz.x, pos3d.xyz.y, pos3d.xyz.z)
    except Exception:
        return None

def blobs_from_mask_fast(mask, min_area):
    num, labels, stats, centroids = cv.connectedComponentsWithStats(mask, connectivity=8)

```



```

blobs = []
for i in range(1, num):
    area = float(stats[i, cv.CC_STAT_AREA])
    if area < min_area:
        continue
    x, y, w, h = stats[i, :4]
    cx, cy = centroids[i]
    blobs.append({
        "bbox": (int(x), int(y), int(w), int(h)),
        "center": (int(round(cx)), int(round(cy))),
        "area": area
    })
return blobs

def detect_color_blobs_optimized(bgr_small, color_ranges, last_pos, last_vel, scale,
use_roi=USE_ROI_TRACK):
    hsv = cv.cvtColor(bgr_small, cv.COLOR_BGR2HSV)
    H, W = hsv.shape[:2]
    out = {c: [] for c in color_ranges}

    dyn_min_area = max(MIN_AREA_ABS_FLOOR * (scale**2), FAR_MIN_AREA_FRAC * (W * H))

    for name, ranges in color_ranges.items():
        mask = None
        for lo, hi in ranges:
            temp_mask = cv.inRange(hsv, lo, hi)
            if mask is None:
                mask = temp_mask
            else:
                cv.bitwise_or(mask, temp_mask, mask)

        if mask is None:
            continue

        mask = cv.morphologyEx(mask, cv.MORPH_OPEN, KERNEL, iterations=1)
        mask = cv.morphologyEx(mask, cv.MORPH_CLOSE, KERNEL, iterations=1)

        submask = mask
        offset = (0, 0)
        pred_center = None
        if use_roi and last_pos.get(name) is not None:
            lx, ly = last_pos[name]
            lx_s, ly_s = int(lx * scale), int(ly * scale)
            vx, vy = last_vel.get(name, (0, 0))
            vx_s, vy_s = int(vx * scale), int(vy * scale)

```

```

        px = int(round(lx_s + vx_s)); py = int(round(ly_s + vy_s))
        pred_center = (px, py)
        roi_r = int(ROI_RADIUS * scale)
        x0, y0, x1, y1 = clamp_roi(px, py, roi_r, W, H)
        submask = mask[y0:y1, x0:x1]
        offset = (x0, y0)

        blobs = blobs_from_mask_fast(submask, dyn_min_area)
        if not blobs:
            continue

        inv_scale = 1.0 / scale
        for b in blobs:
            cx, cy = b["center"]
            x, y, w, h = b["bbox"]
            b["center"] = (int((cx + offset[0]) * inv_scale), int((cy + offset[1]) *
inv_scale))
            b["bbox"] = (int((x + offset[0]) * inv_scale), int((y + offset[1]) * inv_scale),
                        int(w * inv_scale), int(h * inv_scale))
            b["area"] = b["area"] * (inv_scale ** 2)

        best = choose_best_blob(blobs, pred_center)
        if best is not None:
            out[name].append(best)

    return out

# ===== CAPTURE THREAD =====
class OptimizedCaptureThread(threading.Thread):
    def __init__(self, dev_index, wired_mode, delay_us=0):
        super().__init__(daemon=True)
        self.dev_index = dev_index
        self.wired_mode = wired_mode
        self.delay_us = delay_us
        self.device = None
        self.running = False
        self.frame_queue = queue.Queue(maxsize=1)
        self.has_new_frame = threading.Event()

    def open(self):
        cfg = pykinect.default_configuration
        cfg.color_format = pykinect.K4A_IMAGE_FORMAT_COLOR_BGRA32
        cfg.color_resolution = COLOR_RES
        cfg.depth_mode = DEPTH_MODE
        cfg.camera_fps = pykinect.K4A_FRAMES_PER_SECOND_30

```

```

    cfg.wired_sync_mode = self.wired_mode
    cfg.subordinate_delay_off_master_usec = self.delay_us
    self.device = pykinect.start_device(config=cfg, device_index=self.dev_index)
    time.sleep(0.5)

def run(self):
    self.running = True
    while self.running:
        cap = None
        try:
            cap = self.device.update()
            if cap is None:
                continue

            okc, cimg = cap.get_color_image()
            okd, dimg = cap.get_transformed_depth_image()
            if not okc or not okd or cimg is None or dimg is None:
                continue

            cimg_np = cimg.copy()
            dimg_np = dimg.copy()

            frame_data = (cimg_np, dimg_np)

            while not self.frame_queue.empty():
                try:
                    self.frame_queue.get_nowait()
                except queue.Empty:
                    break
            try:
                self.frame_queue.put_nowait(frame_data)
                self.has_new_frame.set()
            except queue.Full:
                pass

        except Exception:
            time.sleep(0.005)
        finally:
            try:
                if cap is not None:
                    cap.release()
            except Exception:
                pass

def get_latest(self):

```

```

        if self.has_new_frame.is_set():
            self.has_new_frame.clear()
            try:
                return self.frame_queue.get_nowait()
            except queue.Empty:
                return None
        return None

    def stop(self):
        self.running = False

# ===== PROCESS & DISPLAY =====
frame_count = 0

def process_and_display_frame(bgra, depth, cam_id, device, last_pos, last_vel, fps,
publisher):
    global frame_count
    frame_count += 1

    H_full, W_full = bgra.shape[:2]
    H_small = int(H_full * DETECTION_SCALE)
    W_small = int(W_full * DETECTION_SCALE)

    bgr_view = cv.cvtColor(bgra, cv.COLOR_BGRA2BGR)
    bgr_small = cv.resize(bgr_view, (W_small, H_small), interpolation=cv.INTER_LINEAR)

    detections = detect_color_blobs_optimized(
        bgr_small, COLOR_RANGES, last_pos[cam_id],
        last_vel[cam_id], DETECTION_SCALE, use_roi=USE_ROI_TRACK
    )

    should_print = (frame_count % 30 == 0)
    timestamp = time.time()

    for name, blobs in detections.items():
        if not blobs:
            continue
        blob = blobs[0]
        (cx, cy) = blob["center"]
        color = DRAW_COLORS[name]

        cx = max(0, min(W_full - 1, cx))
        cy = max(0, min(H_full - 1, cy))

        depth_mm = safe_sample_depth(depth, cx, cy, win=3)

```

```

pos3d_mm = pixel_to_3d(device, cx, cy, depth_mm) if depth_mm > 0 else None

if pos3d_mm is not None:
    Xm, Ym, Zm = pos3d_mm[0]/1000.0, pos3d_mm[1]/1000.0, pos3d_mm[2]/1000.0

    # Publish coordinates with camera ID
    publisher.publish(cam_id, name, Xm, Ym, Zm, timestamp)

    if should_print:
        print(f"Cam{cam_id} {name}: dist={depth_mm}mm, 3D=[{Xm:.3f}, {Ym:.3f}, {Zm:.3f}]m")

    cv.circle(bgr_view, (cx, cy), 6, color, -1)

    if last_pos[cam_id][name] is not None:
        lx, ly = last_pos[cam_id][name]
        last_vel[cam_id][name] = (cx - lx, cy - ly)
        last_pos[cam_id][name] = (cx, cy)

for name in NEON_HUES:
    if not detections.get(name):
        last_pos[cam_id][name] = None
        vx, vy = last_vel[cam_id][name]
        last_vel[cam_id][name] = (int(vx * 0.8), int(vy * 0.8))

cv.putText(bgr_view, f"Camera {cam_id} | FPS: {fps:.1f}", (10, 30), FONT, 0.7, (0, 255, 0), 2, cv.LINE_AA)
cv.imshow("Dual Kinect", bgr_view)

# ===== MAIN =====
if __name__ == "__main__":
    print("Starting optimized dual Kinect tracking with ZMQ publisher...")
    print(f"Detection scale: {DETECTION_SCALE}x (processing at {int(DETECTION_SCALE*100)}% resolution)")
    pykinect.initialize_libraries()

    # Initialize publisher
    publisher = CoordinatePublisher(port=5555)

    cam0 = cam1 = None
    try:
        mode0 = pykinect.K4A_WIRED_SYNC_MODE_MASTER if HAVE_SYNC_CABLE else
pykinect.K4A_WIRED_SYNC_MODE_STANDALONE
        cam0 = OptimizedCaptureThread(0, mode0, 0)
        cam0.open()

```

```

except Exception as e:
    print(f"Failed to open camera 0: {e}")
    raise SystemExit(1)

try:
    mode1 = pykinect.K4A_WIRED_SYNC_MODE_SUBORDINATE if HAVE_SYNC_CABLE else
pykinect.K4A_WIRED_SYNC_MODE_STANDALONE
    delay = SUBORD_DELAY_US if HAVE_SYNC_CABLE else 0
    cam1 = OptimizedCaptureThread(1, mode1, delay)
    cam1.open()
except Exception as e:
    print(f"Camera 1 unavailable: {e}")
    cam1 = None

cam0.start()
if cam1: cam1.start()

last_pos = {0: {name: None for name in NEON_HUES},
            1: {name: None for name in NEON_HUES}}
last_vel = {0: {name: (0, 0) for name in NEON_HUES},
            1: {name: (0, 0) for name in NEON_HUES}}

cv.namedWindow("Dual Kinect", cv.WINDOW_NORMAL)
fps_meter = FPSMeter(60)
display_camera = 0

try:
    while True:
        key = cv.waitKey(1) & 0xFF
        if key in (27, ord('q')):
            break

        frames_processed = []

        pkt0 = cam0.get_latest()
        if pkt0 is not None:
            bgra0, depth0 = pkt0
            frames_processed.append((bgra0, depth0, 0, cam0.device))

        if cam1 is not None:
            pkt1 = cam1.get_latest()
            if pkt1 is not None:
                bgra1, depth1 = pkt1
                frames_processed.append((bgra1, depth1, 1, cam1.device))

```

```

        if frames_processed:
            fps_meter.tick()
            current_fps = fps_meter.fps()

            for bgra, depth, cam_id, device in frames_processed:
                if cam_id == display_camera:
                    process_and_display_frame(bgra, depth, cam_id, device, last_pos,
last_vel, current_fps, publisher)
                    break
                else:
                    bgra, depth, cam_id, device = frames_processed[0]
                    process_and_display_frame(bgra, depth, cam_id, device, last_pos, last_vel,
current_fps, publisher)

            display_camera = 1 if display_camera == 0 else 0

    finally:
        if cam0: cam0.stop()
        if cam1: cam1.stop()

        if cam0: cam0.join(timeout=2.0)
        if cam1: cam1.join(timeout=2.0)

        publisher.close()

    try:
        cv.destroyAllWindows()
    except Exception:
        pass

    try:
        if cam0 and cam0.device: cam0.device.stop_cameras(); cam0.device.close()
    except Exception:
        pass

    try:
        if cam1 and cam1.device: cam1.device.stop_cameras(); cam1.device.close()
    except Exception:
        pass

```

IMU data collection: [imu_data_collection.py](#)

```

"""
This reads the newer sensors at a rate of 400 Hz using the
asyncio and pyserial asyncio modules instead of pyserial, queue, and threading modules
in Python 3.13 / Vizard 8

```

```

"""

import asyncio

import serial
import serial.tools.list_ports
import serial_asyncio

import numpy as np

import struct
import time
import datetime
from time import sleep
import sys
import collections
import os
import csv
import gzip
import ctypes

# SciPy
from scipy.interpolate import PchipInterpolator, CubicHermiteSpline, UnivariateSpline
from scipy.signal import correlate, correlation_lags, butter, filtfilt, find_peaks
from scipy.spatial.transform import Rotation
from scipy.linalg import block_diag
from sklearn.linear_model import LinearRegression

# import viz
# import viztask

##### Define Basic Functions
First #####

def quit_and_disconnect():
    # if (EndofPlay_Flag == False) and (Play_Flag == True) and (Reset_Flag == False) and
    (EndofReset_Flag == True):
        # imu_reset()
        # imu_disconnect()
    # viz.quit()
    sys.exit()

def get_ports():

```



```

ports = serial.tools.list_ports.comports()
return ports

def findArduino(portsFound):

    commPort = 'None'
    numConnection = len(portsFound)

    for i in range(0,numConnection):
        port = foundPorts[i]
        strPort = str(port)

        # look in Device Manager list to see the names of devices connected to USB ports
        # use the first word in the name of the device that you are looking for
        if 'USB' in strPort:
            splitPort = strPort.split(' ')
            commPort = (splitPort[0])

    return commPort

def micros():
    # return a timestamp in microseconds (us)
    tics = ctypes.c_int64() #use *signed* 64-bit variables; see the "QuadPart" variable here:
https://msdn.microsoft.com/en-us/library/windows/desktop/aa383713\(v=vs.85\).aspx
    freq = ctypes.c_int64()

    #get ticks on the internal ~2MHz QPC clock
    ctypes.windll.Kernel32.QueryPerformanceCounter(ctypes.byref(tics))
    #get the actual freq. of the internal ~2MHz QPC clock
    ctypes.windll.Kernel32.QueryPerformanceFrequency(ctypes.byref(freq))

    t_us = tics.value*1e6/freq.value
    return t_us

def millis():
    # return a timestamp in milliseconds (ms)
    tics = ctypes.c_int64() #use *signed* 64-bit variables; see the "QuadPart" variable here:
https://msdn.microsoft.com/en-us/library/windows/desktop/aa383713\(v=vs.85\).aspx
    freq = ctypes.c_int64()

    #get ticks on the internal ~2MHz QPC clock
    ctypes.windll.Kernel32.QueryPerformanceCounter(ctypes.byref(tics))
    #get the actual freq. of the internal ~2MHz QPC clock
    ctypes.windll.Kernel32.QueryPerformanceFrequency(ctypes.byref(freq))

```

```

    t_ms = tics.value*1e3/freq.value
    return t_ms

#-see here for example of constrain function: http://stackoverflow.com/questions/34837677/a-pythonic-way-to-write-a-constrain-function/34837691
def constrain(val, min_val, max_val):
    # "constrain a number to be >= min_val and <= max_val"
    if (val < min_val):
        val = min_val
    elif (val > max_val):
        val = max_val
    return val

#Other timing functions:
def delay(delay_ms):
    # "delay for delay_ms milliseconds (ms)"
    # constrain the commanded delay time to be within valid C type uint32_t limits
    delay_ms = constrain(delay_ms, 0, (1<<32)-1)
    t_start = millis()
    while ((millis() - t_start)%(1<<32) < delay_ms): #use modulus to force C uint32_t-like
underflow behavior
        pass #do nothing
    return

def delayMicroseconds(delay_us):
    # "delay for delay_us microseconds (us)"
    # constrain the commanded delay time to be within valid C type uint32_t limits
    delay_us = constrain(delay_us, 0, (1<<32)-1)
    t_start = micros()
    # % Modulus Divides left hand operand by right hand operand and returns remainder
    while ((micros() - t_start)%(1<<32) < delay_us): #use modulus to force C uint32_t-like
underflow behavior
        pass #do nothing
    return

##### End of Basic Functions #####

# Initialize Vizard
# viz.go()

# Global variables
global count

```

```

n_data_pts = 2400 # Number of data points to collect

idx = 0
count = 0
resync_count = 0
start_time = None

foundPorts = None
connectPort = None

payload_size = 30

# Arrays to store data
imu_meas_time = np.zeros(n_data_pts)
system_time = np.zeros(n_data_pts)
tx_time_stamp = np.zeros(n_data_pts)
accel_x = np.zeros(n_data_pts)
accel_y = np.zeros(n_data_pts)
accel_z = np.zeros(n_data_pts)
omega_x = np.zeros(n_data_pts)
omega_y = np.zeros(n_data_pts)
omega_z = np.zeros(n_data_pts)
accel_x2 = np.zeros(n_data_pts)
accel_y2 = np.zeros(n_data_pts)
accel_z2 = np.zeros(n_data_pts)
accel_hf_1 = np.zeros(n_data_pts)
accel_hf_2 = np.zeros(n_data_pts)
accel_hf_3 = np.zeros(n_data_pts)
status_var_value = np.zeros(n_data_pts)

def is_valid_frame(frame):
    # Check if the frame is valid (status_var = 4095).
    global payload_size
    try:
        status_var = struct.unpack(">H", frame[payload_size - 2:payload_size])[0]
        return status_var == 4095
    except struct.error:
        return False

class SerialReaderProtocol(asyncio.Protocol):
    def connection_made(self, transport):
        self.transport = transport
        self.buffer = b''

```

```

print('Connection established.')
# Send any initialization commands to the microcontroller if necessary

def data_received(self, data):
    global count, start_time, resync_count, payload_size, idx

    self.buffer += data

    while len(self.buffer) >= payload_size and count < n_data_pts:
        frame = self.buffer[:payload_size]
        self.buffer = self.buffer[payload_size:]

        # If the data frame is not valid then the "continue" statement is executed and the
while loop starts
        # over from the top instead of processing the data frame
        if not is_valid_frame(frame):
            resync_count += 1
            self.buffer = b"" # Clear buffer and wait for the next valid frame
            print(f"Resync triggered, count: {resync_count}")
            continue

        # Process the frame
        self.process_frame(frame)

        if count == 0:
            print('SWING NOW!!!')
            start_time = time.perf_counter()
        # else:
        #     print('data')

        count += 1

        if count >= n_data_pts:
            self.transport.close()
            asyncio.get_event_loop().stop()
            break

    def process_frame(self, frame):
        global imu_meas_time, imu_meas_time_hf, imu_collection_time, system_time, payload_size
        global tx_time_stamp, accel_x, accel_y, accel_z, accel_hf_1, accel_hf_2, accel_hf_3
        global omega_x, omega_y, omega_z, accel_x2, accel_y2, accel_z2, status_var_value,
count, idx

        # Get system time

```

```

frame_current_time = time.perf_counter()
system_time[idx] = frame_current_time - start_time if start_time else 0.0

# Unpack the data
integers = []

integers.append(struct.unpack(">L", frame[0:4])[0]) # 4-byte unsigned int

for i in range(4, 22, 2):
    integers.append(struct.unpack(">h", frame[i:i+2])[0]) # 2-byte signed int

for i in range(22, payload_size, 2):
    integers.append(struct.unpack(">H", frame[i:i+2])[0]) # 2-byte unsigned int

idx = count

# Map the data
tx_time_stamp[idx] = integers[0]
accel_x[idx] = -0.009576806641 * integers[1]
accel_y[idx] = 0.009576806641 * integers[2]
accel_z[idx] = -0.009576806641 * integers[3]
omega_x[idx] = -0.002130528872 * integers[4]
omega_y[idx] = 0.002130528872 * integers[5]
omega_z[idx] = -0.002130528872 * integers[6]

accel_x2[idx] = 0.4788403320 * integers[8]
accel_y2[idx] = -0.4788403320 * integers[7]
accel_z2[idx] = 0.4788403320 * integers[9]

accel_hf_1[idx] = 0.4788403320 * integers[10]
accel_hf_2[idx] = 0.4788403320 * integers[11]
accel_hf_3[idx] = 0.4788403320 * integers[12]

status_var_value[idx] = integers[13]

if idx == 0:
    imu_meas_time[idx] = 0.0
else:
    delta_tx = (tx_time_stamp[idx] - tx_time_stamp[0]) if tx_time_stamp[idx] >=
tx_time_stamp[0] else \
    (tx_time_stamp[idx] + 4294967295 - tx_time_stamp[0])
    imu_meas_time[idx] = float(delta_tx / 1e6)

async def main():

```

```

global foundPorts, connectPort

# initialize the com port for data collection
foundPorts = get_ports()
connectPort = findArduino(foundPorts)
print(foundPorts)
print(connectPort)

# Replace 'COM_PORT' with your actual serial port
com_port = 'COM10' # e.g., 'COM10' on Windows or '/dev/ttyUSB0' on Linux
baud_rate = 921600

loop = asyncio.get_running_loop()

# Create serial connection
transport, protocol = await serial_asyncio.create_serial_connection(
    loop, SerialReaderProtocol, connectPort, baudrate=baud_rate
)

"""
if connectPort != 'None':

    if hasattr(transport, "_serial"):
        print("Buffers will be flushed and reset.")
    else:
        print("Transport does not expose the _serial attribute.")
        quit_and_disconnect()

else:
    print('Connection Issue!')
    quit_and_disconnect()

# Flush and reset buffers
serial_connection = transport._serial
serial_connection.reset_input_buffer()
serial_connection.reset_output_buffer()
serial_connection.flush()
"""

if connectPort == 'None':
    print('Connection Issue!')
    quit_and_disconnect()

```

```

try:
    await asyncio.sleep(1e6) # Run indefinitely until data collection is complete -
https://superfastpython.com/asyncio-sleep/
except asyncio.CancelledError:
    pass

# After data collection, save data to CSV
current_time_str = time.strftime("%Y-%m-%d_%H-%M-%S")
file_name = f'../reports/ICM45686_and_ADXL375_test_data_{current_time_str}.csv'

np.savetxt(file_name, np.c_[
    tx_time_stamp[:count], imu_meas_time[:count], system_time[:count],
    status_var_value[:count],
    accel_x[:count], accel_y[:count], accel_z[:count],
    omega_x[:count], omega_y[:count], omega_z[:count],
    accel_x2[:count], accel_y2[:count], accel_z2[:count],
    accel_hf_1[:count], accel_hf_2[:count], accel_hf_3[:count]],
    delimiter=',',
    header='tx_time_stamp,time,system_time,status_var_value,accel_x,accel_y,accel_z,omega_
x,omega_y,omega_z,accel_x2,accel_y2,accel_z2,accel_hf_1,accel_hf_2,accel_hf_3',
    comments='')

imu_idx = np.arange(count)

imu_map = {
    'accel_x':accel_x,'accel_y':accel_y,'accel_z':accel_z,
    'omega_x':omega_x,'omega_y':omega_y,'omega_z':omega_z,
    'accel_x2':accel_x2,'accel_y2':accel_y2,'accel_z2':accel_z2
}

print('Data collection is complete.')
# Run the asyncio event loop
# viztask.schedule(asyncio.run(main()))

asyncio.run(main())

```

Quaternion Coordinates Calculation: [Quaternion Scheme 12_2.py](#)

```
import zmq, time, json, sys
import numpy as np
import os
```

```

#ALWAYS BEGIN TO RUN THIS BEFORE TRACKING SCHEME***** JETSON_IP = "10.132.52.95"
ZMQ_PORT = 5557 ZMQ_ADDR = f"tcp://{JETSON_IP}:{ZMQ_PORT}" ANCHOR_FROM_CAM0 = None
ANCHOR_MIN_POINTS = 3 #require three points

```

----- ZeroMQ setup -----

```
ctx = zmq.Context.instance() sock = ctx.socket(zmq.PULL)
sock.connect("tcp://10.132.52.95:5557") #"tcp://:5557" //laptop: 10.132.22.62

poller = zmq.Poller() poller.register(sock, zmq.POLLIN)
```

----- Output log -----

```
import os
```

Directory where THIS script lives

```
SCRIPT_DIR = os.path.dirname(os.path.abspath(file))
```

One persistent file for all future quaternion outputs

```
OUTPUT_PATH = os.path.join(SCRIPT_DIR, "changing_outputs.jsonl")
```

Open in append mode: keep existing data, add new runs to the end

```
out = open(OUTPUT_PATH, "a")

print("Logging quaternion output to:", OUTPUT_PATH)

np.set_printoptions(precision=6, suppress=True, floatmode="fixed")
```

=====

1) DEFINE FOUR LOCAL (MODEL) COORDINATES HERE (one per color)

=====

```
MODEL = { "neon_yellow": np.array([0.0, 0.03699186, 0.0]),
          "neon_pink": np.array([0.0, 0.01294052, -0.03463435]),
          "neon_green": np.array([-0.02999423, 0.01294052, 0.01731717]),
          "neon_blue": np.array([0.02999422, 0.01294051, 0.01731718]),
        } COLOR_ORDER = ["neon_yellow", "neon_pink", "neon_green", "neon_blue"] # deterministic use-
order
```

2) Build world points dict from an incoming payload.

Returns (W_dict, used_colors):

- W_dict: {color: np.array([x,y,z])} for colors present & finite

- used_colors: ordered list of colors present from COLOR_ORDER

```
def world_from_payload_any(data): pts = data.get("points", []) m = {} for p in pts: try: cid = p["id"] #
e.g., "neon_yellow" pos = p["pos"] x, y, z = float(pos["x"]), float(pos["y"]), float(pos["z"]) if
np.isfinite([x, y, z]).all(): m[cid] = np.array([x, y, z], dtype=float) except Exception: continue

present = [c for c in COLOR_ORDER if c in m]
return m, present
```

3) Horn (Davenport q-method) — works for N >= 3

```
def horn_quaternion(B, W): cb = B.mean(axis=0); cw = W.mean(axis=0) X = B - cb; Y = W - cw M = X.T
@ Y Sxx,Sxy,Sxz = M[0]; Syx,Syy,Syz = M[1]; Szx,Szy,Szz = M[2] N = np.array([ [ Sxx+Syy+Szz, Syz-Szy,
Szx-Sxz, Sxy-Syx], [ Syz-Szy, Sxx-Syy-Szz, Sxy+Syx, Szx+Sxz], [ Szx-Sxz, Sxy+Syx, -Sxx+Syy-Szz,
Syz+Szy], [ Sxy-Syx, Szx+Sxz, Syz+Szy, -Sxx-Syy+Szz ] ], dtype=float) eigvals, eigvecs =
np.linalg.eigh(N) q = eigvecs[:, np.argmax(eigvals)] q = q / np.linalg.norm(q) w,x,y,z = q R = np.array([
[1-2*(yy+zz), 2*(xy - zw), 2*(xz + yw)], [2*(xy + zw), 1-2*(xx+zz), 2*(yz - xw)], [2*(xz - yw), 2*(yz + xw), 1-
2*(xx+yy)] ], dtype=float) t = cw - R @ cb return q, R, t
```

4) Main loop: us

es 4 or 3 visible markers; skips frame if < 3

```
def main(): print("QuatCalc: starting") print(" connecting to {ZMQ_ADDR}") print(f" python:
{sys.executable}") print(f" cwd: {os.getcwd()}") print("FULL OUTPUT PATH:",
os.path.abspath("quat_output_quat.jsonl"))

last_heartbeat = 0

while True:
    events = dict(poller.poll(1000)) # wait up to 1s for a message
    if events.get(sock) == zmq.POLLIN:
        data = sock.recv_json()    print("\nquatchat: received payload")
        camera_id = data.get("camera_id", "cam_0")

        # ---- collect any visible world points for known colors ----
        Wdict, present = world_from_payload_any(data)
        # we want 4 if available, otherwise any 3; skip if < 3
        avail = [c for c in COLOR_ORDER if c in present]
        if len(avail) < 3:
            print(f" only {len(avail)} visible ({avail}) → skipping")
            continue

        #Translation shared origin anchored by cam0
        global ANCHOR_FROM_CAM0 #does not map camera1 on to camera0 like we need, will need
to calibrate when we have our final camera setup in place

        def _mean_world_point(Wd, colors): #get a robust reference position from current points
            arr = np.stack([Wd[c] for c in colors], axis=0)
            return arr.mean(axis=0)

        if ANCHOR_FROM_CAM0 is None and camera_id == "cam_0" and len(avail) >=
ANCHOR_MIN_POINTS:
            ANCHOR_FROM_CAM0 = _mean_world_point(Wdict, avail)
            print(" [shared-origin] anchor set from cam_0:", np.round(ANCHOR_FROM_CAM0, 6))

        if ANCHOR_FROM_CAM0 is not None:
            for c in avail:
                Wdict[c] = Wdict[c] - ANCHOR_FROM_CAM0
```

```

# Build matched sets (use all 4 if present, otherwise the first 3 by COLOR_ORDER)
use_colors = avail if len(avail) >= 4 else avail[:3]
B = np.stack([MODEL[c] for c in use_colors], axis=0) # (N,3)
W = np.stack([Wdict[c] for c in use_colors], axis=0) # (N,3)

# ---- solve pose ----
q, R, t = horn_quaternion(B, W)
print(" used colors:", use_colors)
print(" q (w,x,y,z):", np.round(q, 6))
print(" t:", np.round(t, 6))

# ---- reprojection error (per-point & RMS) ----
pred = (R @ B.T).T + t
errs = np.linalg.norm(pred - W, axis=1)
rms = float(np.sqrt((errs**2).mean()))
print(" per-point err:", np.round(errs, 6), " RMS:", f"{rms:.6f}")

# ---- build processed record for JSONL log ----
record = {
    "camera_id": camera_id,
    "schema_version": data.get("schema_version", None),
    "coord_sys": data.get("coord_sys", None),

    "ts_ns_wall": data.get("ts_ns_wall", None),
    "ts_ns_mono": data.get("ts_ns_mono", None),
    "frame_id": data.get("frame_id", None),

    "anchor_from_cam0": (
        ANCHOR_FROM_CAM0.tolist()
        if ANCHOR_FROM_CAM0 is not None else None
    ),

    "used_colors": use_colors,
    "quaternion_wxyz": q.tolist(),
    "rotation_matrix": R.tolist(),
    "translation_vector": t.tolist(),

    "per_point_errors": errs.tolist(),
    "rms_error": rms,

    "model_points": {c: MODEL[c].tolist() for c in use_colors},
    "world_points": {c: Wdict[c].tolist() for c in use_colors},

```

```

        "raw_payload": data,
    }

    # ---- write the record ----
    out.write(json.dumps(record) + "\n")
    out.flush()

    print("---- waiting for next payload ----")

else:
    now = time.time()
    if now - last_heartbeat >= 1.0:
        print("waiting for payload...", flush=True)
        last_heartbeat = now

if name == "main": main()

```

Angular Velocity Calculation: [angular_velo_computation.py](#)

```

import json
import numpy as np
from scipy.interpolate import PchipInterpolator
import csv

# -----
# Load JSON objects
# -----

t_list = []
quaternions = []

with open("C:\\Users\\Dmitrii Kapranov\\OneDrive - Villanova
University\\Documents\\CAPSTONE\\changing_outputs.jsonl", "r") as f:
    for line in f:
        data = json.loads(line)
        t_list.append(data["ts_ns_mono"])
        quaternions.append(data["quaternion_wxyz"]) # [w, x, y, z]

t_list = np.array(t_list, dtype=np.float64)
quaternions = np.array(quaternions, dtype=np.float64)

# Normalize timestamps to seconds
t_list = (t_list - t_list[0]) * 1e-9

```

```

# -----
# PCHIP interpolators for each component
# -----
quat_itp = [PchipInterpolator(t_list, quaternions[:, i]) for i in range(4)]
quat_dot_itp = [itp.derivative() for itp in quat_itp]

# -----
# Compute angular velocity
# -----
omega_list = []

for t in t_list:
    # Evaluate quaternion and derivative at this time
    q = np.array([itp(t) for itp in quat_itp]) # [w,x,y,z]
    qdot = np.array([itp(t) for itp in quat_dot_itp]) # [w_dot,x_dot,y_dot,z_dot]

    # Renormalize quaternion
    q = q / np.linalg.norm(q)

    # Construct W matrix
    w, x, y, z = q
    W = np.array([
        [-x, w, z, -y],
        [-y, -z, w, x],
        [-z, y, -x, w]
    ])

    # Angular velocity
    omega = 2 * W @ qdot
    omega_list.append(omega)

omega_array = np.array(omega_list) # shape (N,3)

with open('cam_ang_velo.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['time', 'omega_x', 'omega_y', 'omega_z'])
    for t, w in zip(t_list, omega_array):
        writer.writerow([t, w[0], w[1], w[2]])

print(f"\nAngular velocity written to: cam_ang_velo.csv")

```

Cross-correlation: [cross_correlation.m](#)

```

% ECE 4972
% Cross Correlation
% Last Edited: 04/26/2026

```

```

clc; clear; close all;

% Load csv files
% IMPORTANT: Update file path every time you run the code
cam=readtable('C:\Users\Dmitrii Kapranov\OneDrive - Villanova
University\Documents\CAPSTONE\cam_ang_velo.csv');
imu=readtable('C:\\reports\\ICM45686_and_ADXL375_test_data_2025-12-03_14-51-28.csv');

% Access time
t_cam=cam.time;
t_imu=imu.time;

% Access camera angular velocity
wx_cam=cam.omega_x;
wy_cam=cam.omega_y;
wz_cam=cam.omega_z;

% Access IMU angular velocity
wx_imu=imu.omega_x;
wy_imu=imu.omega_y;
wz_imu=imu.omega_z;

% Compute angular velocity magnitudes
cam_mag=sqrt(wx_cam.^2+wy_cam.^2+wz_cam.^2);
imu_mag=sqrt(wx_imu.^2+wy_imu.^2+wz_imu.^2);

% Find delay between streams
[~,~,D]=alignsignals(cam_mag,imu_mag);
t_del=t_imu(D);

% Adjusted camera time by calculated delay
t_adj=t_cam+t_del;

% Plot aligned data streams
figure
hold on
stem(t_adj,cam_mag)
stem(t_imu,imu_mag)
title('Camera and IMU Data Aligned')
xlabel('Time (s)')
ylabel('Angular Velocity Magnitude (rad/s)')

% Calculate % difference between aligned values
imu_alg=imu_mag(D:D+size(t_cam,1)-1,1);
diff=abs(cam_mag-imu_alg);
p_diff=diff./imu_alg;
avg_pdiff=mean(p_diff);

```